William A. Barrett
John D. Couch

COMPILER CONSTRUCTION: THEORY AND PRACTICE

COMPILER CONSTRUCTION Theory and Practice

COMPILER CONSTRUCTION
Theory and Practice

WILLIAM A. BARRETT
JOHN D. COUCH

Composition — Typothetae and William A. Barrett
Acquisition Editor — Alan W. Lowe
Project Editor — Jay Schauer
Technical Art — Blakely Graphics

©1979 Science Research Associates, Inc. All rights reserved.

Printed in the United States of America.

Library of Congress Cataloging in Publication Data

Barrett, William A
Compiler construction.

Bibliography: p.
Includes index.
1. Compiling (Electronic computers) I. Couch, John D., joint author. II. Title.
QA76.6.B367 001.6'425 78–26183
ISBN 0–574–21160–8

10 9 8 7 6 5 4 3 2 1

FOREWORD *by Harold Stone*

Compiler Construction: Theory and Practice is exactly what the title promises. It is an excellent mix of the mathematical foundations of compilers and the practical considerations required in developing high quality compilers for commercial release. The level of discussion is suitable for college juniors and seniors. The material is readily digested because all of the mathematical prerequisites are included in the book, and they are exposed in a highly palatable fashion.

One of the strengths of the book is that algorithms are normally discussed in high-level Pascal-like language that brings out the structure and flow of the algorithms with great clarity in contrast to similar descriptions in flowchart language or automation operations used by some earlier texts in the area. The authors do a particularly commendable job on the practical aspects of code generation, code optimization, and syntax error handling, all three of which are still "black arts" by comparison to the topic of parsing where theory and practice have largely merged.

Here is a book that the professional compiler writer can use to create better language translators, that the computer scientist and engineer can use to gain an understanding and appreciation of the process of language translation that is part of his interface with the computer, and that the student can use to further his knowledge of the capabilities of computers and methods for harnessing the power of the computer.

FOREWORD *by W. M. McKeeman*

The art of translating programming languages has, in many ways, come of age. From its beginnings in Fortran and Algol and hundreds of other languages, we find the concept of high-level languages well established and the construction of translators routine.

Such progress exists because of the highly developed craft of compiler writers to whom the tools of the trade are well-known and regularly applied. There are few places where that rich craft is recorded altogether in a way that is understandable and immediately applicable. To provide such a source is the purpose served by the following text.

The reader must be a programmer. The terminology comes from that field, and the insights necessary to understand the material do so as well. Assuming that background, the reader should find here the world of automatic translation opening up—from the formalities of language description to the tough details of machine code generation. It is a fascinating subject and well worth the intellectual effort of study.

PREFACE

Compiler Construction: Theory and Practice is intended as a one- or two-semester course in the fundamentals of compiler construction and/or language translation. It is designed especially for students with some programming and computer systems background. A strong background in discrete mathematics is helpful, but not required.

This text treats:

- Grammars, trees, and parsing fundamentals.
- Finite-state automata—their relation to regular grammars and regular expressions, their systematic generation, their reduction to minimal form, their representations, and their application to compilers.
- Top-down parsing—principles, LL(k) grammars, LL(1) and recursive-descent parsers.
- Bottom-up parsing—principles, precedence parsers, and the LR(k) parser.
- Syntax-directed translation—principles and application to translation.
- Symbol tables and operations—principles, scope rules, type rules, static representation of PL/I, Cobol, and Pascal structures. Efficient name table access methods.
- Run-time machine models—to support recursive procedure calls, block structure parameter passing, data space allocation, and data access.
- Practical machine systems—a review of three commercial machine systems and their application as a target machine. Details of loader design, support of partial compilation, relocation, recursive calls, etc.
- Optimization—efficient register allocation, constant folding, recognition of common subexpressions, and introduction to data-flow analysis.
- Error recovery—recovering from syntax, scanner and semantic errors, with special attention paid to LR(k) recovery, use of forward move, and experimental studies of recovery.

The Authors

The authors have both academic and industrial experience in compiler construction. William Barrett taught courses in introductory computer systems and compiler construction at Lehigh University, Bethlehem, Pennsylvania,

before accepting a staff position in software development at Hewlett-Packard, Cupertino, California. He is currently responsible for a systems programming language. Its compiler incorporates a number of advanced features described in this textbook. John Couch taught compiler construction at San Jose State University for several years. Formerly responsible for systems software development at Hewlett-Packard, including several compiler projects, he is now Director of Product Development for Apple Computer Co., Cupertino, California.

As teachers, the authors believe that some language theory is an essential part of compiler construction. However, much of language theory is irrelevant to compiler construction. This text should help bridge the gap between theory and practice.

The authors also believe that a balance should be struck between parsing and code synthesis, and between top-down and bottom-up methods. Although most of the synthesis material in this text pertains to bottom-up parsing, both parsing approaches are given approximately equal treatment. Many of the synthesis considerations are applicable to either parsing approach.

Features

Along with basic core material, this text contains source material that has been collected in one book for the first time, and original material found nowhere else.

The first three chapters present a core treatment of grammars, trees, parsing, and finite state automata. We have included only enough language theory and automata theory to understand parsers and compilers. We have omitted such topics as Turing machines and computability. Enough automaton theory is presented to make it possible to develop the commonly used top-down and bottom-up parsers, and to prove that they perform as claimed. We believe that, with the formal descriptions, proofs, extensive discussions, and the many examples of machines and machine traces, a student will be able to understand how a parser works, and will also be able to understand professional literature in programming languages for additional information.

Chapter 3 discusses finite state automata, their relation to regular grammars and regular expressions, and their reduction to minimal form. As an application, lexical analysis is discussed briefly. A designer must be aware that some languages, such as Fortran, pose difficult lexical problems. Such problems are discussed in some detail.

The remaining chapters contain much original material. An extensive discussion of recursive descent parsers will be found in Chapter 5. We not only describe this commonly used parsing method, but also develop an automatic generator and the conditions under which the parser operates correctly.

The LR(k) parsing methods, which are recently being incorporated into some commercial compiler systems, are discussed in considerable detail in Chapter 6. Methods of reducing the size of the stored LR tables and of estimating the size of the tables are also given. Parser reductions based on special grammar properties are described in some detail.

The organization of a symbol table for single and multiple pass compilers with multiple block levels is given in Chapter 8. Efficient access methods are also described there. We show how to organize a symbol table for PL/I structures and Pascal types, and how to resolve partially specified names.

Run-time data structures are dealt with in Chapter 9. We develop a special machine system that can be used to implement Algol, Fortran, Pascal, and many other languages. This system, or its equivalent, must be emulated or simulated in order to support various features of these languages. The important issues of procedure calls, parameter passing, blocks, and array and structure access are dealt with in considerable detail. The material on the efficient access of multi-dimensional arrays and structures should be of particular interest.

Three specific machine architectures and their supporting system conventions are described in some detail in Chapter 10: the HP 3000, CDC 6400, and the IBM 360. The HP 3000 system is described at length. However, the principles of loader table design and machine architecture, exemplified in the HP 3000, are applicable to any system. In particular, we show how procedures can be independently compiled and brought together later by a loader to form a complete program.

Optimization issues are dealt with in Chapter 11. We have followed a modern school of thought in this area—that of constructing a directed acyclic graph of a program segment, then performing various reductions on that graph. We consider the multi-register allocation problem at some length, following the work of Aho and others in this field. Finally, we develop the fundamental notions of flow analysis, which can be used to reduce code and to detect subtle programmer errors during compilation.

The book ends with a chapter on error recovery. We classify program errors, then deal at some length with the problem of patching over a syntax error in a free-form language. We give different methods of dealing with syntax errors and discuss their effect on semantic issues. We summarize the results of some error recovery experiments.

Designing a Compiler Course

For a one-semester course, some portions of this text should be covered in detail; other sections should be approached tangentially. To concentrate on basic matters in one semester, Chapters 1–4 and 6–9 provide a solid grounding in grammars, top-down and bottom-up parsers, syntax-directed translation, symbol table issues, and static and dynamic data structures. We sug-

gest skipping Chapter 5, on precedence methods, and Chapters 10 through 12.

For a two-semester course, use Chapters 1–7 and Chapter 12 for the first semester, then 8–11 for the second semester. These chapters should be augmented with additional outside material or a class project. Many suggestions for projects are contained in exercises in these chapters. If the course is being taught for the first time, a useful project would be a top-down or bottom-up parser generator, based on the algorithms given in the text. This generator can then be used in subsequent years as a tool for practice in designing grammars and in writing compilers.

A simulator of the system described in Chapter 8 and a compiler that generates code for this system would also be instructive projects.

Acknowledgements

We wish to thank Frank DeRemer, Harold Stone, and Bill McKeeman for many helpful criticisms and a detailed reading of the manuscript; Richard Page of Hewlett-Packard for simulating the AOC machine and correcting potential errors; Steve Glanville for his criticism and for permission to use his thesis material; Tom Pennello for permission to use his thesis material; George Miller of the University of California for classroom testing this text; and Fred Clegg, and Tom Whitney for their patience, encouragement, and understanding.

We wish to thank Hewlett-Packard for allowing us to use their computer systems for manuscript preparation. We thank Control Data Corporation, IBM, and Hewlett-Packard for permission to create illustrations based on their technical manuals.

Finally, we thank our wives who supported us, despite the fact that we were around the house much less than they (and we) would have liked, through the many long hours spent working on "The Book."

W. B.
J. C.

CONTENTS

Foreword by Harold Stone *v*

Foreword by W. M. McKeeman *vi*

Preface *vii*

1. Introduction **1**

1.1. Translators 1
1.2. Why write a compiler? 2
1.3. The cost of a compiler 6
1.4. The compiling process 7
1.5. Translator issues 11

2. Introduction to language theory **15**

2.1. Language elements 15
2.1.1. Tokens and alphabets 15
2.1.2. Strings 16
2.2. Generative grammars and languages 17
2.2.1. Terminals and nonterminals 18
2.2.2. Production rules and grammars 19
2.2.3. Classes of grammars 20
2.2.4. Sentential forms and language definition 26
2.2.5. Production trees and syntax trees 28
2.2.6. Canonical derivations 40
2.2.7. Ambiguity 43
2.3. Introduction to parsing 46
2.3.1. Top-down and bottom-up parsing 46
2.3.2. Backtracking 50
2.3.3. A deterministic top-down parser 58
2.3.4. A deterministic bottom-up parser 60
2.4. Bibliographical notes 62

3. Finite-state machines **63**

3.1. Formal definitions 67
3.2. Transformation of a NDFSA to a DFSA 75
3.2.1. Empty cycle detection and removal 76
3.2.2. Removal of empty transitions 77

3.2.3. Transformation from nondeterministic to deterministic 82
3.2.4. Accessible states 85
3.3. Machine equivalence 88
3.3.1. Definitions 88
3.3.2. Reduction 90
3.3.3. A systematic reduction method 94
3.4. Regular grammars and FSA 97
3.5. Regular expressions and FSA 100
3.5.1. Definitions 100
3.5.2. Regular expression identities 103
3.5.3. Correspondence to FSA 104
3.5.4. Regular expression of a regular grammar 111

3.6. FSA representations 114

3.6.1. Sparse array tables 114
3.6.2. Table reductions 117
3.6.3. Sparse array representation of a FSA 117
3.6.4. Program representation of a FSA 120
3.7. Applications of FSA 122
3.7.1. Recognition of literals 122
3.7.2. Lexical analysis 124
3.8. Some FSA theorems and their proofs 132
3.8.1. Equivalence of empty cycle states 132
3.8.2. Equivalence through removal of empty moves 132
3.8.3. Equivalence on the NDFSA to DFSA transformation 134
3.8.4. The pairs table reduction algorithm 134
3.9. Bibliographical notes 136

4. Top-down parsing 137

4.1. Nondeterministic push-down automata 137
4.2. LL(k) grammars 148
4.2.1. Definitions 148
4.2.2. Some properties 149
4.3. Deterministic LL(1) parser 155
4.3.1. LL(1) selector table 156
4.3.2. LL(1) grammar transformations 160
4.4. Recursive descent parsers 161
4.4.1. Construction and validation 167
4.4.2. Extended grammars 176
4.4.3. Construction and validation from extended grammar 178
4.5. Bibliographical notes 192

5. Bottom-up parsing and precedence parsers 193

5.1. Nondeterministic bottom-up parsing 193
5.2. Precedence parsing 199
5.2.1. Relations 201
5.2.2. Boolean matrix sum and product 202
5.2.3. Viable prefix 205
5.2.4. Precedence pairs 205
5.2.5. Precedence relations 206
5.2.6. Simple precedence grammar 206
5.2.7. Wirth-Weber relations 212
5.2.8. Other precedence parsers 218
5.3. Bibliographical notes 223

6. Bottom-up LR(k) parsers 224

6.1. LR(k) grammars and parsers 224
6.1.1. LR(k) grammars 224
6.1.2. An LR(1) parser 225
6.1.3. LR(0) parser construction 235
6.1.4. Resolution of inadequate states 246
6.2. LR(k) parsers 252
6.2.1. Canonical LR(1) parsing tables 254
6.2.2. A canonical parser 255
6.2.3. Table reductions 260
6.2.4. LALR(1) tables 268
6.3. Augmented grammars 269
6.4. Size of LR(k) tables 273
6.5. Comparison of parsing methods 275
6.6. Bibliographical notes 278

7. Syntax-directed translation 279

7.1. General principles 279
7.1.1. Definitions 280
7.1.2. Tree transformations 282
7.2. Simple SDTS and top-down transducers 284
7.3. Simple postfix SDTS and bottom-up transducers 289
7.4. A general transducer 295
7.5. String transducers and their limitations 299
7.5.1. String translators 299
7.5.2. Abstract-syntax tree construction 303
7.5.3. A practical bottom-up synthesis system 308
7.6. Bibliographical notes 319

8. Static representations of data objects 320

8.1. Symbols and declarations 321
8.2. General organization of a symbol table 323
8.2.1. Scope of names 324
8.2.2. Names and attributes 329
8.3. Data objects and their static representation 333
8.3.1. Primitive objects 334
8.3.2. Types 337
8.3.3. Structures 342
8.3.3.1. Array objects 344
8.3.3.2. PL/I structures 347
8.3.3.3. Pascal structures 352
8.4. String tables and their access 361
8.4.1. Linear access 363
8.4.2. Binary access 364
8.4.3. Tree access 366
8.4.4. Hash access 368
8.4.5. Comparison of access methods 372
8.5. Bibliographical notes 375

9. Run-time machine structures 376

9.1. Introduction 376
9.2. Run-time structures for Algol-like languages 377
9.2.1. Arithmetic and logical expressions 379
9.2.2. Assignment statements 382
9.2.3. Conditionals 384
9.3. Stack and heap allocation 386
9.4. Input-output 388
9.5. Blocks and storage allocation 388
9.6. Procedures and recursion 393
9.6.1. Procedures and the free variable problem 397
9.6.2 Textual addresses 401
9.6.3. Block entry and exit 403
9.6.4. The static display chain 407
9.6.5. The display revisited 411
9.6.6. Labels and GOTO's 413
9.7. Arrays 418
9.7.1. Packed arrays 418
9.7.2. Array access through matrix pointers 425
9.7.3. Dynamic arrays and redimensioning 429
9.7.4. Pascal data structures 431
9.7.5. Pascal data object access 434
9.8. Typed procedures and procedure parameters 437

9.8.1. The procedure copy rule (PCR) and call by name 438
9.8.2. Call by value (CBV) 439
9.8.3. More about free variables 441
9.8.4. Fortran parameter-passing rules 442
9.8.5. Implementation of CBV and typed procedures 443
9.8.6. Typed procedure return value 445
9.8.7. Implementation of CBN procedure parameters 446
9.8.8. Implementation of CBN label parameters 448
9.8.9. Summary of procedure parameter mechanisms 450
9.8.10. Call by name of implementation 454
9.8.11. CBV versus CBN 460
9.9. Summary of AOC instructions 460
9.10. Bibliographical notes 453

10. Object code and machine architectures 465

10.1. Introduction 465
10.2. Intermediate languages 466
10.3. The pros and cons of intermediate languages 470
10.4. Machine architectures 473
10.5. The Hewlett-Packard 3000 475
10.5.1. Data formats 475
10.5.2. Memory and register organization 476
10.5.3. Memory reference instruction format 478
10.5.4. Indirection and indexing 479
10.5.5. Stack configuration during program execution 482
10.5.6. Stack marker, procedure calls, and exits 482
10.5.7. Instructions 487
10.5.8. Allocation of memory space for OWN and outer block variables 493
10.5.9. Partial compilation and segmentation 493
10.5.10. The USL file structure 494
10.5.11. Procedure compilation and USL linkage 496
10.5.12. Primary DB header 497
10.5.13. Secondary DB/OWN initial values header 498
10.5.14. Procedure call header 499
10.5.15. OWN variable pointer correction header 501
10.5.16. Procedure local variables 501
10.5.17. Branches and constants 503
10.5.18. Conclusions 507
10.6. The Control Data 6000 computer system 511
10.6.1. Registers and arithmetic 513
10.6.2. Instruction format 515
10.6.3. The register set operations 516

10.6.4. Arithmetic and logical operations 517
10.6.5. Branches 518
10.6.6. Other operations 520
10.6.7. Procedure parameters in a Fortran implementation 520
10.6.8. Arithmetic expressions 521
10.6.9. Saving and restoring registers 522
10.6.10. Relocatable linking and partial compilation 523
10.6.11. A Pascal implementation 524
10.6.12. Summary 525
10.7. The IBM System/360 527
10.7.1. Instructions 529
10.7.2. An instruction example 531
10.7.3. Procedures and program relocation 534
10.7.4. Save areas 535
10.7.5. Object modules 536
10.7.6. Addressing 538
10.7.7. Object module design 539
10.7.8. Register allocation 541
10.7.9. Summary 542
10.8. A generalized code generator 543
10.8.1. Table construction 547
10.8.2. Experimental results 549
10.9. Bibliographical notes 550

11. Optimization 551

11.1. Machine-dependent optimizations 553
11.2. Machine-independent optimizations 553
11.2.1. Expression (AST) optimizations 556
11.2.2. Flattening 557
11.3. Optimal AST evaluation for a multiregister machine 562
11.3.1. The machine 563
11.3.2. Tree labeling 564
11.3.3. Optimal code generation 565
11.3.4. Discussion 567
11.3.5. Commutative operators 570
11.3.6. Associative and commutative operators 571
11.4. Code improvement over a sequence of statements 572
11.4.1. Blocks 573

11.4.2. Variables and their domains 574
11.4.3. Equivalent and normal blocks 575
11.4.4. Representation of a block as a (DAG) 576
11.4.5. Value of a DAG 577
11.4.6. Common subexpression identification 578
11.4.7. Use of associativity and commutativity 579
11.4.8. DAG reduction 580
11.4.9. DAG evaluation 581
11.4.10. Register assignment and code generation 584
11.5. Data flow analysis 587
11.5.1. Definitions 588
11.5.2. The basic data flow analysis method 594
11.5.3. Intervals 596
11.5.4. Higher order intervals 597
11.5.5. Use and live information 599
11.5.6. The interval-based reach algorithm 600
11.5.7. Applications of data flow information 603
11.6. Bibliographical notes 605

12. Error recovery 607

12.1. Introduction 607
12.2. Semantic errors 611
12.3. Syntax errors 613
12.3.1. General methods 614
12.3.2. Diagnosis of a syntax error 614
12.3.3. Patching a syntax error 615
12.3.4. Semantics operations in error recovery 618
12.3.5. A bounded-range error recovery strategy 618
12.3.6. Variations on the bounded-range strategy 620
12.3.7. Forward move 621
12.3.8. Correction strategies with a forward move 623
12.3.9. Empirical study of error recovery 628
12.3.10. Recovery in a recursive descent compiler 636
12.4. Bibliographical notes 637

Annotated bibliography 639

Index 655

CHAPTER 1

INTRODUCTION

1.1. Translators

A *translator* accepts a *source program* and transforms it into an *object program*. The source program is a member of a *source language* and the object program is a member of an *object language*. Both languages are artificial, inasmuch as they are designed for a digital computer, as opposed to a natural language like English or German.

Each program expresses some algorithm. We are primarily interested in those translations for which the source and object algorithms are identical. For example, a Fortran program should yield the same results for a given input regardless of the machine language to which it is translated. Those results should be as expected from the specification of the Fortran language and the algorithm as expressed in Fortran.

Artificial translation is rapidly becoming a mathematical discipline, while natural translation remains rather more an art. Yet the two are somewhat akin. Any student of foreign languages knows that one language cannot be translated to another by simply substituting words. A human translator must first grasp the precise meaning of each source sentence, then compose an equivalent sentence in the object language. So it is with artificial translators. A source program must first be analyzed to uncover its underlying meaning and structure; this process is called *parsing*. Then a number of transformations on the structure are performed, ultimately ending in the object program.

For a given source language, the translation may be carried to several different levels of completeness: to *assembly code*, to *machine code*, or to execution. Assembly code is a sequence of mnemonic instructions and symbolic address references; it is a member of an *assembly language*, and must be translated into machine code by yet another translator called an *assembler*. Assembly language usually has a very simple structure with a fixed format—a program location field, an instruction field, and an address field. Each line of assembly code usually translates to one machine instruction. There are no nested statements, arithmetic expressions, or procedures as in Fortran or Algol.

Machine code is a sequence of binary machine instructions that require little or no modification in order to be executed.

An *interpreter* accepts a source program, translates it into some intermediate data structure, then executes the algorithm by carrying out each operation given in the intermediate structure. An interpreter is considerably less efficient than a compiler, because it carries the burden of intermediate structure analysis as well as execution. However, a program can be rapidly

developed with an interpreter, since its test can follow its modification so rapidly.

The advantage of a compiler is that it generates an efficient, short, executable program. It demands fairly heavy computer resources while compiling, but when executing, only those resources needed by the executing program are required. The disadvantage of a compiler is the lag time between writing a program and executing it.

An interpreter as an algorithm is very similar to a compiler. The analysis of the source statements and bookkeeping for identifiers and literals are tasks common to interpreters and compilers. We shall therefore not be concerned with the differences between an interpreter and a compiler in this textbook.

A compiler or an interpreter is itself a program written in some language, called the *host* language. We therefore see that three languages are involved in a compiler– source, object, and host. These are often three different languages. A Fortran compiler that runs on an IBM 360 might be written in PL/I, and generate machine code for a 1401. A compiler that generates code for its host machine is called *self-resident*; if, in addition, it is written in its own source language, it is *self-compiling*. If it generates code for a machine other than the host, it is called a *cross compiler*.

1.2. Why Write a Compiler?

A programming language is designed and a compiler written for it for only one reason–to make it easier for human beings to get a computer to carry out a class of tasks. If it were possible for us to rapidly translate a task description into the long lists of binary numbers that a computer expects as its instruction list, with no errors, then programming languages and compilers would be unnecessary. Unfortunately, human beings make mistakes. They are unable to cope with a big list of binary numbers, and their time is valuable compared to machine time. A computer is well suited to clerical tasks and can handle them cheaply and accurately. One task that a computer can be expected to perform is assisting us in programming itself.

Here are some of the services that a compiler should provide for its user:

1. Evaluate symbolic references to instructions and instruction locations. Consider branches. At the machine level, a branch might look like this in 16-bit octal code:

location	contents
037663	024433

On an HP 2100 minicomputer, the number 024433 is interpreted as a direct branch to location 433. (The "024" is the instruction and the "433" is the address.)

Now it is unreasonable to expect anyone to remember the rather complicated pattern of bits that make up a branch instruction, nor those of the

dozens of other instructions offered on a typical small computer. It is therefore better to assign a mnemonic name such as JMP to each instruction. Then the same branch might be written

location	contents
037663	JMP 433

This change reads better than the 024433, yet now requires some kind of translator, one which can interpret the characters in "JMP" and turn them into the "024" of the JMP instruction on a 2100. The other instructions may similarly be given mnemonic codes. A simple program might then look like this:

	location	contents	comment
(start)	0	LDX 7	{Load index register, address 7}
	1	LDA 7,X	{Load accumulator, indexed}
	2	DSZ	{Decrement X and skip if zero}
	3	JMP 6	{Go to location 6}
	4	ADA 10,X	{Add location 10, indexed, to accumulator}
	5	JMP 2	{Go to location 2}
	6	HLT	{Stop}
	7	4	{Data needed by the program}
	10	16	
	11	32	
	12	176	
	13	24	

This program can now be read by someone familiar with the 2100 instruction set. The intention of the programmer was to add up the list of four numbers in locations 10 through 13. However, the program doesn't do that; it simply loads 24 in the accumulator register, then halts. There are several errors; a correct program is given below:

location	contents
0	LDX 6
1	LDA 7,X
2	ADA 6,X
3	DSZ
4	JMP 2
5	HLT
6	3
7	16
10	32
11	176
12	24

Now these errors would be obvious only to someone with considerable experience in 2100 assembly language coding. Yet this is quite a simple algorithm. How many errors are likely to be introduced in the assembly-language coding of a large data base manager or of an operating system?

Notice that lots of things have changed between the two programs. The most painful change is the removal of one instruction, which has caused a shift in the locations of the variables and all the instructions past the deleted one. In other words, every instruction referring to one of the shifted constants must be changed. One simple change in a long list of instructions can require many changes throughout the program.

The burden of finding and changing lots of instructions can be removed as a human activity and shifted to a computer by introducing symbolic location names. Any location that must be referred to in an instruction is assigned a symbolic name; then only names need appear in instructions. The assembler must then determine the location of each label and fix the instructions accordingly. The sample program then might look like this:

```
      LDX   L1
      LDA   L2,X
L4:   ADA   L1,X
      DSZ
      JMP   L4
      HLT
L1:    3
L2:   16
       32
      176
       24
```

Symbolic labels carry several unexpected bonuses. The absolute locations have disappeared, which means that our program may now be placed anywhere within some other program; the assembler will work out the locations. Only those locations that must be referenced are labeled; however, the label associated with some instruction may be hard to find in a large listing. The assembler may check that every symbol appears as a label exactly once in the program.

2. Constants should be converted to internal form by a compiler. Modern computers can handle a variety of internal data forms, for example, multiple-precision integers, floating-point (real) numbers, packed decimal numbers, strings, etc. No human being should be expected to perform the required conversion to internal form; there is also no valid reason why a programmer should even have to know the internal form.

3. Special control structures may be devised that read better than the primitive instructions of an assembler. For example, the sample program might read as follows in Algol:

```
INTEGER ARRAY A(4):=(16, 32, 176, 24);
INTEGER I, SUM;

SUM := 0;
FOR I := 0 UNTIL 3 DO SUM := SUM + A(I);
```

Although this is somewhat more wordy than the assembly language form, it is certainly easier to understand. The data variables are clearly declared separately from the algorithm that operates upon them, and the one-line FOR statement expresses the desired operation very clearly.

If a concise description of an algorithm is the most desirable feature of a programming language, then the language APL probably takes top honors. The above algorithm to add the four numbers 16, 32, 176, and 24 is written this way in APL:

```
SUM ← +/(16 32 176 24)
```

4. Programs written in any of several common high-level languages are often transportable. By this, we mean that a program written in, say, Fortran, can be compiled and executed with few changes on any of several different computers, despite differences in internal architecture and instruction set. When software is transportable, a computer user with a large program library may change computers without incurring a heavy software development cost. Programs written in assembly language for one computer are worthless on any other computer. Computers also become obsolete, and the assembly language programs written for them become worthless.

5. Assembly language programs are much more likely to contain subtle errors than the same programs written in a high-level language. There are several reasons for this: assembly language is hard to read except by an expert very well versed in the machine; assembly language programs are "unstructured," i.e., there are no control structures to guide coding and reading; it takes many more assembly language instructions to achieve the same effect as a suitable high-level language statement; and a machine instruction often has many subtle side-effects to trap an unsophisticated programmer, e.g., a condition code or carry bit may be set or cleared and an index register value may be altered. A modern high-level programming language and its compiler should protect the programmer from many such error-causing complications.

6. A high-level language lends itself to the division of the labor of a software task. A programming team can agree on the properties of certain procedures or macros, then go their separate ways to write and check out their part of the whole task. This principle also applies to a sophisticated assembler, but more agreements need to be reached on programming conventions among the team during the design phase.

7. A modern high-level language will engender good programming style. For example, an IF-THEN-ELSE construct forces a programmer to

consider both alternatives of an IF test; a failure to deal with one cries out from the printed page. In assembly language, it is often difficult to find the two alternatives of a branching test, and it is easy to obscure one of the two. The compiler can be made to check array bounds, so that during testing, any instance of an out-of-range index can be detected and subsequently analyzed. Good programming style also means that someone else can read and understand the program without a lot of analysis.

8. A number of high-level languages are rather simple in structure and can easily be learned by someone with little or no computer background. For example, Basic is used extensively in some grade schools. Such languages make computer services accessible to people who would otherwise never consider using them.

These are some of the major advantages provided by a high-level programming language and a compiler for it. There are many others. All these result in an appreciable increase in engineering efficiency in the writing, maintenance, and modification of software.

1.3. The Cost of a Compiler

The development of a compiler is a major software effort. Depending on the complexity of the language and the target machine, as little as three man-months or as much as thirty man-years may be required to write and debug a compiler. The most complex compiler ever written was probably PL/I for the IBM 360. PL/I is an extraordinarily rich language, containing not only several file access methods, but a large set of data types and operations.

Another cost is a certain loss of machine efficiency for a program written in a high-level language compared to the same algorithm written by a skilled programmer using assembly language. A high-level language imposes certain constraints upon a programmer in its forms of control structures, limited data types, etc., which do not exist in assembly language. This loss of efficiency is particularly severe for a high-level language that is not particularly well suited to its target machine. Thus Algol and PL/I are rather well suited to a stack architecture. A multiregister machine, such as the 360, generally requires elaborate optimization techniques in order that a compiler can compete with an assembler in number of words of code and execution time. The optimization phase of compiler design for a machine poorly suited to the language can double the compiler cost and size.

A compiler's inefficiency in generating executable code is paid for upon each use of the code. If that cost is deemed too high, several alternatives can be chosen, among them recoding in another language, or coding portions of the software in assembly language. Often an inefficiency in a programming system stems from a poor choice of algorithm, or a poor peripheral device access strategy, rather than from an inherent inefficiency in the compiler.

1.4. The Compiling Process

The major operations in a compiler are illustrated in figure 1.1. The process begins with a source file at the top of the figure, and ends with optimized object code at the bottom. Our description in this section will necessarily be highly simplified; many special problems in a real compiler system will be overlooked in this review.

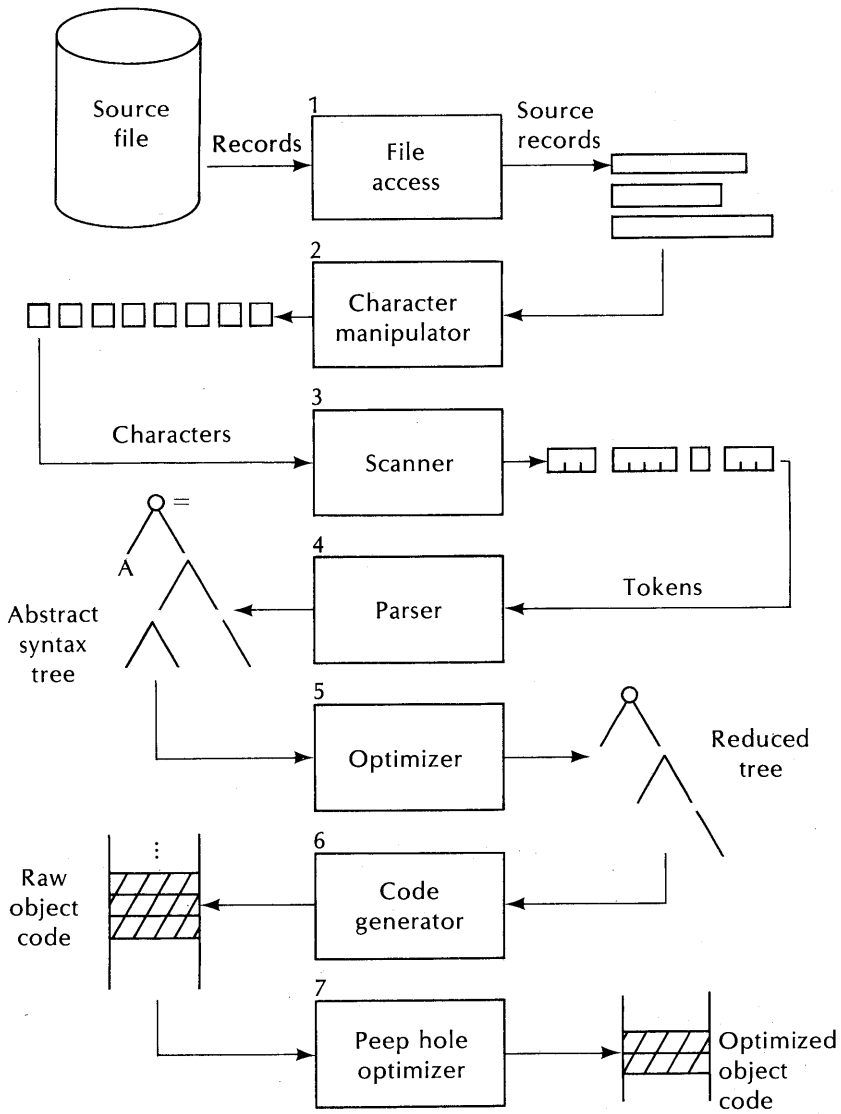

Figure 1.1. Major operations in a compiler.

A compiler is based on a sequence of transformations that preserve the operational meaning of a program, but not necessarily all the information in it, nor even the exact sequence of operations requested (or thought to be requested) in the source program.

The nature of the system upon which the compiler resides has a strong influence on the design of a compiler. Most computers have a severely limited "fast" memory (semiconductor or magnetic core), but extensive "mass" memory (disk, drum, magnetic tape). A compiler is often expected to process very large source programs, so that only a relatively small portion of the source can be actively under process at any one time. A compiler must usually be such that only the least information necessary be retained in fast memory, and such that most of the compilation is sequential in character–object code or some intermediate structures will be emitted as additional source is read.

Not all translators fit this pattern, nor must a compiler be strictly sequential on all systems. A system with virtual memory, for example, has unlimited memory available, in effect, so that sequential processing is less important. An interpreter usually has to carry all of the source, symbol tables, and program along during editing and execution; however, they need not necessarily all be in fast memory.

Compilation begins with some source form, shown in figure 1.1 as a file. Of course, source can originate in any of a variety of forms, such as punched cards, paper tape, a terminal, and magnetic tape. Their access is quite different, but the differences are usually of little or no concern to the compiler writer. File access (box 1) is generalized in most operating systems, so that the particular form of source is of no consequence. The compiler therefore first sees some sequence of source records, emitted by box 1.

In some languages, source record boundaries are important as statement delimiters (e.g., Fortran or Basic). In others, source boundaries are of no consequence. Hence a compiler will likely contain a section that accepts source records and emits a sequence of characters, box 2. This section may well detect and remove comments and special control commands that have nothing to do with the source language. If the language specification is such that blanks are ignored, then blanks would be suppressed by the character manipulator, box 2. However, this is not always easy. In Fortran, blanks are crucial in some contexts but not in others, and the necessary distinctions are sometimes difficult.

The character sequence is next subdivided into a sequence of *tokens* by a *scanner* or *screener*, box 3. A token may be a single character, or some special sequence of characters. The scanner may be responsible for skipping comments and blanks, if necessary. Examples of tokens are identifiers (names assigned to variables, statement labels, etc.), quoted strings, numbers, and special character sequences, such as ":=" in Algol.

Boxes 2 and 3 are sometimes called a *lexical analyzer*. They must be tailored to the language and the grammatical description of the language

chosen by the compiler implementors. For example, the Fortran source record

```
6      DOI       =4,X *( Y-16)       ,16
```

would be translated by a lexical analyzer into the token sequence:

```
6
DO
I
=
4
,
X
*
(
Y
-
16
)
,
16
```

The token sequence emitted by the lexical analyzer is next processed by a *parser*, box 4, whose task is to determine the underlying structure or "meaning" of the program. Until the parser is reached, the tokens have been collected with little or no regard to their position within the program as a whole. The parser considers the context of each token and classifies groups of tokens into larger entities such as *declarations*, *statements*, and *control structures*. The product of the parser is usually an *abstract syntax tree*. (An example of an abstract syntax tree for the Fortran DO statement is given in figure 1.2.) A tree is a very useful representation of a program or program segment, inasmuch as it facilitates several subsequent transformations designed to minimize the number of machine instructions needed to carry out the required operations.

The abstract syntax tree may next be subjected to a number of optimizations through a tree optimizer, box 5. The result is some reduced tree, possibly rearranged to suit the needs of the machine architecture. Examples of optimization possible at this level include: constant expression evaluation, use of commutativity, associativity, and distributivity of certain operators to collect constant expressions together, detection of common subexpressions, or rearrangement to suit a particular architecture.

The reduced tree may then be transformed into a sequence of raw object code by a code generator, box 6. The object code may be of several different

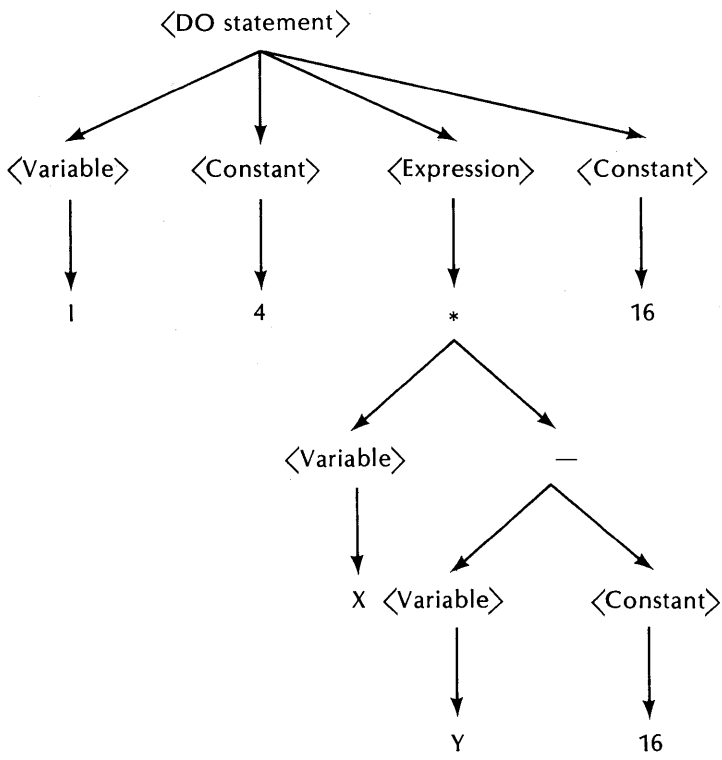

Figure 1.2. Abstract syntax tree for the Fortran statement DO I=4, X*(Y−16), 16

kinds, depending on the purpose of the compiler: (1) it may be machine code for some particular target machine, or (2) it may be a special intermediate file designed to be further processed by a loader, or (3) it may be specially designed intermediate code that must be further translated by another system into machine code or a loader structure.

Finally, the raw object code may be subjected to further optimizations by a *peep-hole optimizer*, box 7. Such an optimizer examines short sequences of code and determines whether, in certain cases, a sequence can be replaced by a shorter equivalent sequence. For example, the sequence

```
STOR  A;  {copy accumulator to location A}
LOAD  A;  {copy location to accumulator}
```

could be reduced by removing the LOAD instruction. This sequence can easily occur if the STOR represents the end of one statement and LOAD the beginning of another. As another example, the sequence

```
LOAD  A;
ADD   =1;{add constant 1 to accumulator}
STOR  A;
```

might be replaced by a single instruction,

```
INCM  A;  {increment contents of location A}
```

on a machine containing such an instruction. Such optimizations can appreciably reduce the number of emitted instructions.

Other operations not shown in figure 1.1 include *string table management* and *error recovery*. The string table contains a copy of each identifier appearing in the program. It is usually linked to an *attribute table*, so that properties assigned to some identifier can be made available throughout the compilation process. The string table and attribute table together are called a *symbol table*. The attributes of an identifier might include:

- How it was declared, (e.g. REAL, ARRAY).
- Its dimensions, if an array.
- The number of storage elements required for it.
- Its location in memory.

An *error recovery* system is usually attached to the parser, box 4, and takes control when a syntax error in the source program is detected. Its purpose is to diagnose the error and attempt some kind of recovery, so that the subsequent source input need not be discarded. The error recovery system may, for example, decide to insert some tokens, or discard some input tokens in an attempt to patch over the offending section of source.

1.5. Translator Issues

Often a very simple compiler can be made by omitting all the optimization steps, even the tree-building step. Many compilers generate machine code directly through the parse steps, using a variety of heuristic methods to achieve the translation.

A compiler might translate a source language to another closely-related intermediate language, or to symbolic assembly code. The latter can then be translated by some existing compiler or assembler to machine code. Many of the functions of a full compiler can thus be omitted, considerably reducing the compiler development task. However, such a compiler will likely neither be very efficient nor generate particularly efficient object code.

A compiler that generates an intermediate language could be used with several different machines. Of course, a translator from intermediate lan-

guage to each machine's code is also needed, but these are often easier to write than several different complete compilers. For example, a translator from Pascal to PCODE has been used to implement Pascal on several different machines. In some of the implementations, PCODE is interpreted, rather than translated.

The steps outlined in figure 1.1 may be carried out in one or several *passes*. A pass in general is some scan through either the source records, or through some translation of the source. A practical multipass compiler requires sufficient temporary memory to hold the intermediate translation; this could be any read/write medium. The intermediate structures may well be much larger than the final machine code, and in any case, during compilation, the machine's memory must be shared with the compiler program and the compiler's data areas.

In a one-pass compiler, the different sections of the compiler represented in figure 1.1 appear as different procedures in one large compiler program. Whenever a point in some process is complete, a procedure may be called that accomplishes the next step. For example, the parser may be the primary "driver" for the compiler, i.e., the main program, first called. It might then call the lexical analyzer to deliver one token; the lexical analyzer in turn may call a file handler to deliver one record, etc. The parser in turn might then call a tree-builder as the various parts of a source statement are analyzed. The tree could be built in one of several different ways, but when enough is completed, a tree optimizer might be called, then it calls the code emitters. In such a system, all of the steps of figure 1.1 are repeatedly performed as the compiler scans the source text, and code is emitted in short segments that correspond to segments of source text.

In a multipass compiler, each of the steps in figure 1.1 might be a separate pass. In practice, some groups of steps are combined, for example, it is feasible to construct a sequence of abstract syntax trees in one pass, then devote subsequent passes to reducing or transforming the tree. The tree itself can either be stored as a linked-list data structure, or represented in some linear form, such as reverse Polish.

The principle advantage of a multipass compiler is its ability to collect information that can be used to efficiently allocate storage for variables and emit instructions, information that is often difficult to obtain in one pass. Some optimizations require several scans over a major source program module. The principle disadvantage is that it is only applicable to a computer system with sufficient intermediate storage. Small computer systems that have very poor or slow intermediate storage (e.g. paper tape) are therefore poor candidates for a multipass compiler.

Summary

A compiler, as a translator of one language to another, can be organized in a variety of ways, depending on the source language, the target language, the

degree of optimization desired, the time available to develop the compiler, and its future value. A characteristic of nearly every compiler is a means of translating source records into a sequence of tokens (characteristic of the language), parsing this sequence to yield a syntax tree, and then transforming the tree to yield the object program. Alternatively, it is usually possible to bypass tree building, and simply emit code directly from the parsing system. Modern practice seems to favor tree construction and multiple passes, because they facilitate a number of optimizations, modularize the compiler design, and are often more efficient than the alternatives.

For Discussion

1. Choose some assembler you are familiar with and make a list of the services that it provides its users. How valuable are each of these services? Are there services that it could provide, but fails to?

2. Consider two or three high-level programming languages that you are familiar with and discuss those features that you feel are most valuable to a programmer. Also discuss features that you would most like to see added to the language.

3. How desirable is brevity in a programming language? (Compare Cobol and APL, for example.) Discuss the merits and demerits of extreme brevity.

4. What do you feel should be uppermost in the design of a "good" programming language among the following characteristics? How much does your choice depend on the application or the user community? Which desirable features are likely to conflict with each other?

(a) Ease of grasping the program's algorithm, upon reading the program.

(b) Ease of learning the programming language for the first time.

(c) Protection against coding errors, to the degree possible in a language.

(d) Ability to use any feature supported by the target machine, as needed, in order to obtain the most efficient execution.

(e) Convenience to the keypunch or terminal operator.

(f) The number of different operations available in the language.

(g) Immediate line-by-line syntax checking while preparing the program.

(h) A large number of operations.

(i) Conciseness of expression. (Compare APL and Cobol.)

5. Suppose that you are given an assignment of writing a compiler for some language to be implemented on a machine equipped with a symbolic assembler and a Fortran compiler. Discuss in general terms your strategy, given each of the following objectives:

(a) The compiler may implement only selected features of the language L (your choice) and may be very inefficient, but must be finished as soon as possible.

(b) The compiler must implement the complete language, may generate inefficient code, but must run as efficiently as possible and require as little memory space as possible. (It will be used for short student programs that will likely be executed only once).

(c) The compiler may be as large as you have time and patience to develop it and may be inefficient as a program, but must generate the most efficient code possible for the target machine.

(d) The compiler may generate inefficient code and be inefficient as a program, but must later be transformed into a compiler for another target machine with minimum effort.

CHAPTER 2

INTRODUCTION TO LANGUAGE THEORY

2.1. Language Elements

The elements of a language are its *alphabet*, its *grammar*, and its *semantics*. The *alphabet* is a finite set of *tokens* of which sentences in the language are composed. The *grammar* is a set of structural rules defining the legal contexts of tokens in sentences. The *semantics* is a set of rules that define the operational effect of any program written in the language when translated and executed on some machine. This and the next four chapters deal principally with grammars and translation apart from semantics.

Sentences in English are constructed from the set of characters consisting of the letters, digits, space, and punctuation marks. These characters are composed into words through the aid of spelling rules and a dictionary, and then words are composed into sentences through the aid of grammatical rules.

A computer program may similarly be constructed from a sequence of characters drawn from the computer's character set. Such a sequence may be composed into tokens (corresponding to English words) and the tokens composed into sentences through a grammar. The principal difference between English and a programming language is that the grammatical and spelling rules for English are very complicated and have many exceptions and ambiguities, while the corresponding rules for a programming language are concise, highly structured, have few (if any) special cases, and (hopefully) no amibiguities.

In this chapter, we will develop the notions of alphabets, strings in an alphabet, generative grammar systems, and the problem of *parsing* a sentence. Parsing is the process by which we determine the specific grammar rule applications that yield a given sentence; it corresponds roughly to determining the structure of an English sentence.

2.1.1.Tokens and Alphabets

An *alphabet* is a finite set of *tokens*, fixed at the time of definition of the source language. Every source program consists of some sequence of tokens drawn from the alphabet of the source language.

The alphabet of a programming language could be a set of keyboard characters; each letter, each blank, each digit, and special symbol is a distinct token, and it is possible to define useful programming languages on such an alphabet.

More often, certain easily recognizable sequences of characters are collected together to form the tokens of the language. For example, in Algol 60, the characters ":" and "=" when written together in that order ":=" represent one token which is used for assignments. In Fortran, the character sequence DO usually represents a special token that initiates a loop structure. The task of assembling a sequence of source characters into tokens comprises that part of a compiler called the *scanner* or *lexical analyzer*. That task may be simple or difficult; the end result in any case is to yield a sequence of tokens for the benefit of the language structural analysis that is to follow.

Examples of alphabets:

1. The Fortran character set, which contains the 26 letters, the ten digits, and the special symbols $+ \; - \; * \; / \; . \; , \; (\;) \; = \; \$ \; ' \; :$, a total of 48 characters. Every Fortran program may therefore be considered as one long string of characters. For practical reasons, the end of a Fortran statement and a blank may also be considered characters, and are important as language elements. The end of a Fortran program, or end-of-file, may also be considered a character in the alphabet, bringing the alphabet to 51 characters.

2. The set of Fortran special names, identifiers, constants, and special symbols. In this alphabet, special names such as DO, IF, and READ are separate tokens, distinct from names invented by the programmer. Similarly, a number is considered a token. A name invented by a programmer (an identifier) is considered an element of the alphabet.

The symbol Σ will be used to designate an alphabet. Mathematically, Σ is a set of objects. Lower case letters near the beginning of the English alphabet, e.g., a, b, c, d, will usually represent members of Σ.

2.1.2. Strings

A *string* is a sequence of elements drawn from an alphabet. The set of all strings of length one or more consisting of members of Σ will be designated Σ^+. Thus if an alphabet Σ consists of the characters #, 4, %, and +, written:

$$\Sigma = \{\#, 4, \%, +\}$$

then each of these strings is a member of Σ^+:

#

4444# #%+

+# #4

+++

There are, of course, many other possible members.

The length of a string is the number of characters in the string, written:

$$|\text{FORTRAN}| = \text{length of the string "FORTRAN"} = 7$$

Thus $|4444\#\ \#\%+|$ is 8, $|\#|$ is 1, etc.

Lowercase letters near the end of the alphabet, e.g. w, x, y, will be used to represent strings.

A string whose length is zero is called the *empty string*, written ϵ.

Two strings, w and x, may be connected together to form a single new string, wx. This operation is called *concatenation*. Either of the strings may consist of a single character, multiple characters, or the empty string. Concatenation is an implied operation and as such has no special symbol. The concatenation of an empty string with any other string x results in x; that is:

$$\epsilon x = x$$

$$x\epsilon = x$$

$$x\epsilon y = xy$$

Note that if strings x and y are each members of Σ^+, then the string xy is also a member of Σ^+.

Although the empty string "disappears" when concatenated with any other string, it could be a member of any set of strings. In particular, the set Σ^* is the set Σ^+ together with the empty string, which can be written using the set union operator $\cup$ as:

$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

In terms of compilers, an empty string corresponds to a source program containing no tokens. In practice, such a program must be followed by an end-of-file indicator. The compiler then can conclude that the program is an empty string by detecting the end-of-file as the first token.

An empty string is not the same as an empty set $\varnothing$. A set may consist only of the empty string; such a set is not empty. An empty set contains nothing, not even an empty string. Again, there is a correspondence of these concepts in a compiler. A compiler may accept an empty string by noticing an end-of-file marker. A compiler that accepts an empty set will reject every input including an empty input and will then halt regardless of the input source.

2.2. Generative Grammars and Languages

A *language* is some subset of Σ^*, where Σ is its alphabet. This definition is much too broad to be of any practical use. If the alphabet consists of the English characters, then Σ^* contains the text of all the books in the world, but

also contains whatever happens to be pecked out by a monkey at a typewriter. Σ^* includes meaningful sentences of value to a reader, but also includes random sequences of characters.

A language may also be finite or infinite. If a language is infinite, there cannot be a bound on the maximum length of a string in the language. If there were such a bound, then because the alphabet is finite, there is a finite number of arrangements of the tokens in the finite-length strings.

To be useful, a language must contain structure. There must be a reasonably small set of rules that govern the manner in which the tokens of Σ may be organized into strings in the language. We call the set of rules that define a legal class of strings a *grammar*.

For example, telephone numbers in the United States have a certain structure familiar to everyone—an area code, an office number, a party number, and an extension, for example:

(212) 438-7021 X643

Some of these components may not appear in a phone number; if there are no extensions or the area code is understood, they may be omitted. Also some telephone systems permit dialing only the party number. We can represent a phone number by the structure

<area code> <office> <party> <extension>

Each of these has a structure of its own. The area code is commonly written in parentheses, e.g., (212), so we see that the structure <area code> has the structure

(<three-digit number>)

and in turn, this structure <three-digit number> has the structure

<digit> <digit> <digit>

Finally, each <digit> has one of the forms 0, 1, 2, . . . , 9.

The overall structure of a phone number can be seen as a set of structures, each of which provides a more detailed description of the number. The "smallest" components are the members of the phone number alphabet, consisting of the digits, dashes, parentheses, and "X".

2.2.1. Terminals and nonterminals

A *terminal* is any member of Σ, and is therefore a synonym for token. A *nonterminal* stands for some set of strings in Σ^* , but is not itself in Σ. Nonterminals are used in a language's structural rules. For a given grammar, there will be a finite set of nonterminals; this set will usually be designated N. A *symbol* is a terminal or a nonterminal.

In the telephone number structure example given above, each of the names

<area code>
<office>
<party>
<extension>

are nonterminals. We could equally well use single characters as nonterminals, and shall do so frequently. For most of the simple example grammars in this text, we will use capital letters near the beginning of the English alphabet to designate a particular nonterminal, e.g. A, B, C. A capital letter near the end of the English alphabet, e.g. X, Y, Z, will generally stand for an "arbitrary" nonterminal.

For large grammars, we will need more than 26 nonterminals, and will therefore fabricate special nonterminal names. The convention of using < > to designate a nonterminal is used widely to define programming languages. The nonterminals <area code> and <office> are examples of this convention.

2.2.2. Production Rules and Grammars

A *production rule*, or *production* for short, has the general form

$$x \rightarrow y$$

where x and y are strings in the terminal and nonterminal sets of a given grammar. The y may the empty string, but x cannot be. Productions are used in a special subclass of grammars called *replacement systems*. Essentially, in such a system, we can generate other strings in the language by starting with a string of the form

wxz

then applying a production of the form $x \rightarrow y$ to yield a new string

wyz

We have effectively replaced x in the string wxz by y, through a given production $x \rightarrow y$. The y is called a *simple phrase* of wyz. The replacement step wxz to wyz is called a *derivation step*, and we say that wxz *derives* wyz.

Eventually, given enough such replacements and productions, we could end up with a string of terminal symbols. However, we haven't yet specified where wxz came from, nor have we restricted the productions $x \rightarrow y$ in any way.

If we could start with any string at all, and then commence with replacement rules, we would really have failed to define a language. One possibility is "no replacement," so such a strategy leads to a language

consisting of Σ^* again. We therefore need some uniquely specified starting string. We may in fact choose one nonterminal as the starting string, rather than some other string w, without loss of generality, since we can always add a production of the form $S \rightarrow w$ to a grammar whose starting string is w.

Next we need some precise way of knowing when to stop a sequence of derivations. We choose to stop when we obtain a string that contains no nonterminals; the nonterminals after all are subject to further reduction. This feature implies two other important properties that a grammar should satisfy: (1) the nonterminal and terminal sets are disjoint. If a token is both terminal and nonterminal, it is not clear whether to stop the derivation or not. (2) Every production rule must contain at least one nonterminal in its left member, i.e., given a production $y \rightarrow x$, y must contain at least one nonterminal. If both properties are satisfied by the grammar, then no further derivations are possible once we have an all-terminal string.

We conclude that a grammar is a four-tuple (Σ, N, P, S), where Σ is a terminal alphabet, N is a nonterminal alphabet, Σ and N are disjoint, P is a set of productions of the form $y \rightarrow x$, where y and x are in $(N \cup \Sigma)^*$, and y contains at least one element in N, and S is a designated start symbol in N.

Such a grammar is called a *phrase-structure* grammar.

2.2.3. Classes of Grammars

Chomsky [1965] distinguished four general classes of grammars. The most general class, the *unrestricted* grammars, is not phrase-structured. The other three classes are phrase-structured. They are the *context-sensitive*, *context-free* and *right-linear* grammars.

The most general phrase-structured class is the *context-sensitive* grammar. In this class, each production has the form

$$x \rightarrow y$$

where x and y are members of $(N \cup \Sigma)^*$, x contains at least one member of N, and $|x| \leq |y|$. Note that the latter requirement implies that y cannot be empty. An example of a context-sensitive grammar is

$$G_1 = (\{S, B, C\}, \{a, b, c\}, P, S)$$

where the productions P are

1. $S \rightarrow aSBC$
2. $S \rightarrow abC$
3. $CB \rightarrow BC$
4. $bB \rightarrow bb$
5. $bC \rightarrow bc$
6. $cC \rightarrow cc$

Let us develop a set of replacements in this grammar. Since S is the designated starting string, we look for a production with S as its left member. Either of the first two will do:

$$S \rightarrow aSBC$$

so that aSBC is a new string. Now in string aSBC, we can only use another S rule; let's choose the second one:

aSBC becomes aabCBC

Here, the third or fifth rule may be chosen; let us choose the third:

aabCBC becomes aabBCC

Continuing, we find the following sequence of replacements:

aabbCC

aabbcC

aabbcc

We end up with all terminals, so this is the end of the possible replacements.

We could reach a string for which no production can apply. For example, in the string aabCBC, if we choose the fifth rule instead of the third, we obtain aabcBC, and we find that no rule can be applied to this string. The consequence of such a failure to obtain a terminal string is simply that we must try other possibilities until we find those that yield terminal strings.

Exercises

1. Derive the strings

   abc
   aaabbbccc

   in G_1: $(\{S, B, C\}, \{a, b, c\}, P, S)$, where $P = \{S \rightarrow aSBC, S \rightarrow abC, CB \rightarrow BC, bB \rightarrow bb, bC \rightarrow bc, cC \rightarrow cc\}$.

2. Show informally that the strings

   abbc
   aabc
   abcc

   cannot be derived in G_1.

3. From Exercises 1 and 2, frame a conjecture regarding the kind of strings derivable in G_1, and attempt an informal proof of your conjecture.

4. Derive a context-sensitive grammar for strings of 0's and 1's such that the number of 0's and 1's is equal.

Context-Free Grammars

The next most general class of grammars is one that we shall be studying in most of this text—the class of context-free grammars. In a context-free grammar, or CFG, each production has the form $x \rightarrow y$, where x is a member of N, and y is any string in $(N \cup \Sigma)^*$. Note that y may be the empty string, hence any context-free grammar with a rule $A \rightarrow \epsilon$ cannot be context-sensitive; the latter class does not permit such a rule.

An example of a context-free grammar is one that we shall be using repeatedly, an arithmetic expression grammar G_0:

$$N = \{E, T, F\}, \Sigma = \{+, *, (,), a\}, S = E, \text{ and } P = \text{the set}$$

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow a$

Here, the nonterminal set is clearly {E, T, F}, the terminal set is {+, *, (,), a}, and the start symbol is E. We may obtain a typical expression by applying the replacement rules, as before:

E derives E + T, using the first rule
E + T derives T + T, using the second rule
T + T derives F + T, using the fourth rule
F + T derives a + T, using the last rule
a + T derives a + F, using the fourth rule
a + F derives a + a, using the last rule

Many other examples of derived terminal strings in this grammar may be obtained.

Whenever a grammar has several productions with the same left member, we will sometimes use the symbol "|" which stands for *alternation*. Thus the two rules $E \rightarrow E+T$ and $E \rightarrow T$ may also be written

$$E \rightarrow E+T \mid T$$

Exercises

1. Derive the following strings in G_0:

   a*a+a
   (a+a)*a
   ((a))
   a*a*a

2. Show informally that the following strings cannot be derived in G_0:

   a*+a
   a+aa
   ((a)

3. Show informally that in G_0:

   (a) Any string of the form (...(a)...) can be derived, where the number of left parentheses is equal to the number of right parentheses.

   (b) Any string of the form a+a+a+a+...+a can be derived.

   (c) The pairs *+, ++, **, and +* can never appear in a derivation.

   (d) The pair aa can never appear in a derivation.

4. Show informally that each of the nonterminals in G_0 can derive an arbitrarily long terminal string.

5. Give an example of a simple grammar containing a nonterminal that cannot derive any terminal string, i.e., where every derived string contains some nonterminal.

6. Construct a context-free grammar for telephone numbers, along the line introduced in section 2.2.

7. Write a context-free grammar for the following language. The notation 1^n stands for a sequence of n 1's.

   $$01^n01^n0 \text{ where } n \geq 0$$

   Examples of strings in this language are: 000, 01010, 0110110.

8. Write a context-free grammar that specifies the set of decimal literals that may be written in Fortran. Examples of these literals are

   −21.5
   0.25
   3.7E−6
   .5E7
   6E6
   100.E+3

   Note that E or a decimal point is sufficient to specify a decimal number. If neither is present, then the number is considered fixed point.

9. Given the grammar G = ({A, B, C, D}, {x, y}, P, A), where P is the set of productions

   A → B | D
   B → BCC | x
   C → yx
   D → xCyD | xy

   show that x, xy, xyxyx, and xyxyxyxyxy but none of xyx, xyxy, and xyxyxyx are derivable from A.

Right-Linear Grammars

If each production in P has the form A → xB or A → x, where A and B are in N and x is in Σ^*, the grammar is said to be *right-linear.*

The right-linear grammars are clearly a subset of the context-free grammars. An example of a right-linear grammar is the following grammar G_2; it defines a set of ternary fixed point numbers, with an optional plus or minus sign:

V → N | +N | −N
N → 0 | 1 | 2
N → 0N | 1N | 2N

Two other grammars related to the right-linear grammar are the *left-linear* and the *regular* grammar. A left-linear grammar has productions in P of the form A → Bx or A → x, where A, B, and x have the above meanings. A regular grammar is such that every production in P, with the exception of S → ϵ (S is the start symbol) is of the form A → aB or A → a, where a is in Σ. Further, if S → ϵ is in the grammar, then S does not appear on the right of any production.

An example of a regular grammar here defines the fixed point decimal numbers with a decimal point. The "d" stands for a decimal digit:

$$
\begin{aligned}
S &\rightarrow dB \mid +A \mid -A \mid .G \\
A &\rightarrow dB \mid .G \\
B &\rightarrow dB \mid .H \mid d \\
G &\rightarrow dH \\
H &\rightarrow dH \mid d
\end{aligned}
$$

Exercises

1. Derive the following strings in G_2:

   220
   −12
   +2

2. Show informally that the following strings cannot be derived in G_2:

   +
   2−0
   3+

3. Show informally that G_2 can derive arbitrarily long strings. What property of the grammar makes this possible?

4. Give a left-linear grammar that expresses the same set of terminal strings as G_2.

Significance of the Grammar Classification

These grammar classifications are to some extent arbitrary. One may define many variations on the basic patterns given. However, these particular definitions lead to particularly simple classes of sentence recognizing machines or *automata*.

An *automaton*, for our purposes, is some system with a finite description (but not necessarily containing a finite number of parts) that can accept some string of terminal symbols, given a grammar, and that can determine whether the string can be derived in the grammar.

The process of finding a derivation, given a grammar and a terminal string supposedly derivable in the grammar, is called *parsing*, and an automaton capable of parsing is called a *parser*. A parsing automaton is clearly of value in a compiler. A grammar is a concise, yet accurate description of some

language; it expresses the class of structures that we want in the language. However, so far we see only how to construct legal strings in the language. We need to solve the opposite problem: given some string, to determine if it is legal. We also need to go farther than that; we must determine the sequence of productions needed to obtain the string.

Now each of the three phrase-structured grammar classes has a fairly simple yet powerful automaton associated with it:

1. The right-linear grammars can be recognized by a finite-state automaton, which consists merely of a finite set of states and a set of transitions between pairs of states. Each transition is associated with some terminal symbol. We shall define finite-state automata more completely in the next chapter.

2. The context-free grammars are accepted by a finite-state automaton controlling a push-down stack, with certain simple rules governing the operations. The push-down stack is the only element that can be indefinitely large. However, only a finite group of top stack members are ever referenced in the finite description of this automaton.

3. The context-sensitive grammars are accepted by a two-way, linear bounded automaton, which is essentially a Turing machine whose tape is not permitted to grow longer than the input string.

Of these three, only the first two will be dealt with at length in this textbook. It turns out that the class of context-free grammars is sufficiently powerful to encompass most of the features of nearly every common programming language. Those features which are not covered by a context-free grammar are not in practice covered by a context-sensitive grammar, either, but require special extensions to the recognizing automaton.

2.2.4. Sentential Forms and Language Definition

Recall that in one derivation step, we transform a string

$$wxz$$

into a string

$$wyz$$

given a production $x \longrightarrow y$ in the grammar. We represent a derivation step by the symbol $\Rightarrow$,

$$wxz \Rightarrow wyz$$

Each of the strings wxz and wyz are called *sentential forms*, provided that we started with the start symbol S of the grammar and obtained wxz through a sequence of derivation steps.

A sequence of one or more derivation steps is indicated by

$$\Rightarrow^+$$

For example, in grammar G_0, we have

$$E \Rightarrow E+T \Rightarrow E+T+T \Rightarrow T+T+T$$

hence we may write

$$E \Rightarrow^+ T+T+T$$

Here, since E is the start symbol, the string $T+T+T$ is a sentential form in G_0.

A sentential form consisting only of terminals is called a *sentence*. Clearly, if we have

$$S \Rightarrow^+ (\text{some sentence})$$

then we have obtained, or *derived* a member of the language defined by the grammar. We may put this in set notation as follows.

Given a grammar $G = (N, \Sigma, P, S)$, then L(G), the language defined by G, is

$$L(G) = \{x \mid x \in \Sigma^* \text{, and } S \Rightarrow^+ x\}$$

The form $\{x \mid C\}$ is read: "the set of all x's such that condition C holds." L(G) is then the set of all strings x such that x is terminal and x can be derived from the start symbol S.

This definition holds for any of the three language classes context-sensitive, context-free, and right-linear.

Under this definition, L(G) may consist of the null set, or it may consist of one or more terminal strings, possibly including the empty string. For example, consider the grammars G_E and G_O, as follows:

$$G_E = (\{S\}, \{\epsilon\}, \{S \rightarrow \epsilon\}, S)$$

$$G_O = (\{S\}, \{\epsilon\}, \{S \rightarrow S\}, S)$$

In neither case is the production set empty (it could be, incidentally). However, in the first case, we have a language consisting of one string, the empty string. Such a language might be accepted by a compiler that notices that a source program contains nothing, and then halts (or in practice, goes on to the next source program in a job sequence).

In the second case, G_O, the only production just yields another S and can never yield a terminal string. Nor can it yield the empty string; we need some way for $S \Rightarrow^+ \epsilon$ to do that. We conclude that the language of G_O is empty ($\varnothing$), and is different from the language of G_E.

Sometimes it is useful to refer to a derivation of "zero" steps. This just means "no step"; the sentential form is left unchanged. We indicate a derivation of zero or more steps by the symbol $\Rightarrow^*$.

Exercises

1. Show that the following are sentential forms in G_0:

   (E+a)
   a+T*a
   (E)*a

2. Given a grammar with an empty production set, is its language empty?

3. Can a grammar have an empty nonterminal set? An empty terminal set? A terminal set consisting only of the empty string? If so, what can be said of the grammar's language in each case?

2.2.5. Production Trees and Syntax Trees

Recall that a tree is useful as an intermediate representation of a program or a portion of a program. It is also useful as a means of representing a derivation of some sentence in a context-free language, or of representing the productions of a CFG.

A *tree* is an abstract representation of a certain connectedness among a set of objects. The objects are called *nodes* and the connections among them are called *directed edges*. A tree may be constructed of distinct objects by the following recursive process:

1. One distinguished node is called the *start node*. Let N designate a start node; N is a tree.

2. Given any node N of a tree T with no out-directed edges. We may construct another tree T′ from T by adding one or more nodes $N_1, N_2, \ldots, N_n$ (not already in T) to T, and connecting each of these to node N by an edge directed from N to the node. The nodes $N_1, N_2, \ldots, N_n$ are called the *children* or *immediate descendants* of N, and N is called the *parent* or *immediate ancestor* of the nodes $N_1, N_2, \ldots, N_n$. The nodes $N_1, \ldots, N_n$ are *siblings* of each other.

Every tree has exactly one node with no indirected edges called the *root* node. A node with no outdirected edge is called a *leaf* or *terminal* node. Every tree has at least one terminal node that may also be the root. A node with at least one outdirected edge is called an *internal* node. A *path* is any set of nodes $n_1, n_2, \ldots, n_k$ such that one edge connects n_i to n_{i+1}, in that order, for all i such that $1 \leq i < k$.

The length of some path containing n nodes is $n-1$. For the sake of generality, we consider one node as a path of length zero.

Given any node N, there is a unique path from the root to that node. If its length is L, then node N is said to be at *level* L in the tree.

There is no path that connects a node to itself. A tree is said to be *acyclic* for this reason, however, there are acyclic graphs that are not trees.

The *height* of a tree is the maximum level in the tree, hence the length of the longest path. This longest path must extend from the root to some leaf node.

We will generally draw our trees upside down, with the root node at the top. figure 2.1 shows a tree with its parts labeled.

A tree may be embedded in a plane with each node a distinct point and no two edges crossing. The tree definition suggests how this can be done.

Each node N of a tree is the root of another tree, sometimes called a *subtree* rooted in N.

A tree embedded in a plane can be ordered in several different ways. The scheme we shall most often use is called a *preorder*, or *left-to-right natural order*, illustrated in figure 2.2. We obtain a preorder by imagining the tree surrounded by a directed circle, with the direction counterclockwise (figure 2.3). Then let the circle collapse around the tree, so that by following its path we contact every edge twice, once on one side and once on the other. We then order the nodes by assigning 1 to the root and the successive integers to the nodes when first touched by the collapsed circle.

A *postorder* is obtained by following the same procedure as a preorder, except that successive integers are assigned to the nodes when *last* touched by the collapsed circle.

Any tree can be represented in a linear memory space by a set of nodes, each of which contains two pointers: to its child and to its right sibling. Since either or both of these may not exist, a special pointer called a *nil* pointer is needed, that indicates this fact. Such a pointer system also imposes an ordering on the children. However, it is difficult to find the parent node of a given tree node N with this system; it is necessary to examine all the nodes in the tree starting with the root until we find that node one of whose children is N.

In order to locate a parent node rapidly, we may either add another pointer to each node, pointing to its parent, or set the right sibling pointer of each right-most sibling to point to its parent. The latter kind of tree is shown in figure 2.4. We need only some mark on the right-most sibling to indicate that its right sibling pointer points to the parent, not to its right sibling.

A Pascal data structure for such a tree is the following:

```
type TREENODE: record CHILD, SIBLING: ↑ TREENODE;
                      PARENT: Boolean
               end
```

The symbol ↑ means that CHILD and SIBLING are pointers to a data structure of type TREENODE. If PARENT is TRUE, then SIBLING is the right-most sibling, and it points to its parent. The links in figure 2.4 clearly are in preorder.

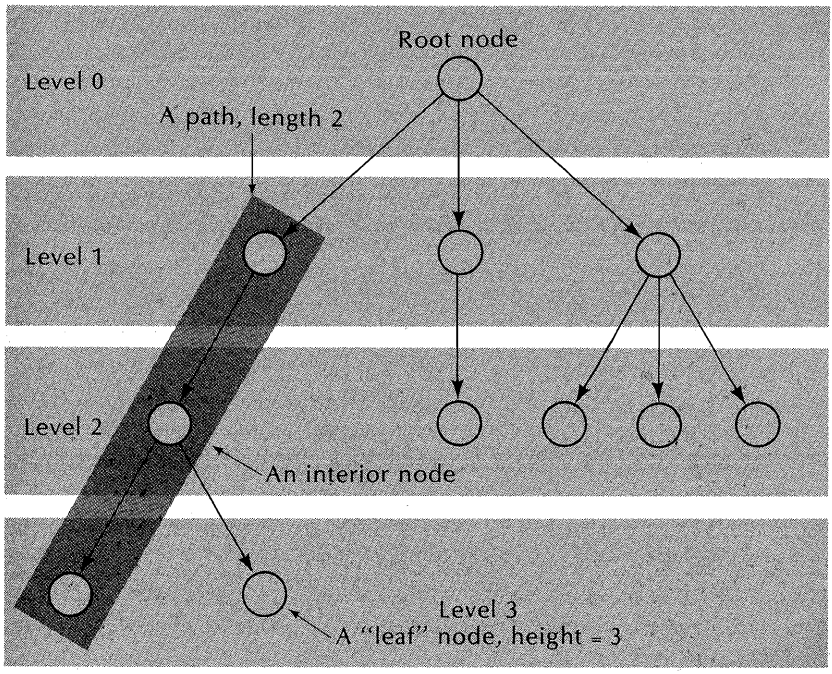

Figure 2.1. A typical tree.

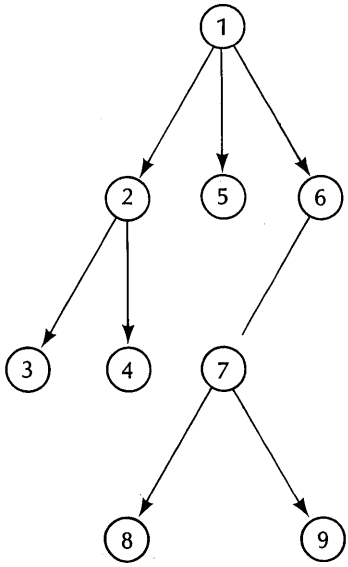

Figure 2.2. Preorder.

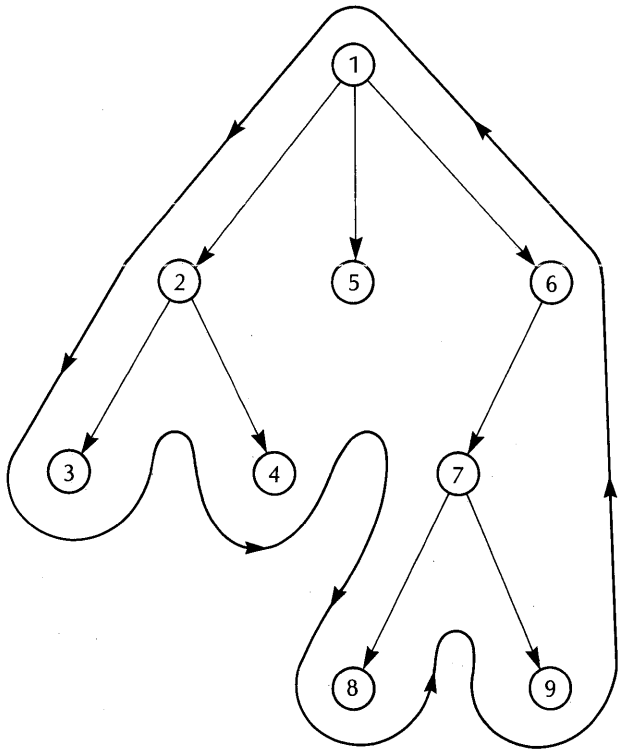

Figure 2.3. Obtaining preorder.

A tree node is said to be *decorated* when it carries some information in addition to its connectness. We may attach any sort of data to a node. In practice, we simply add more cells to each of the node elements shown in figure 2.4. For example, a cell may contain some simple data element or a pointer to some other data structure.

Exercises

1. The terminology of tree structures is obviously borrowed from certain properties of the plant phylum. If the root system of a plant must be included, is there a correspondence of the plant's components to a tree? Why not?

2. Consider the biblical injunction, "No man can serve two masters" (Matt. 6:24). If this applies to a business organization, would the

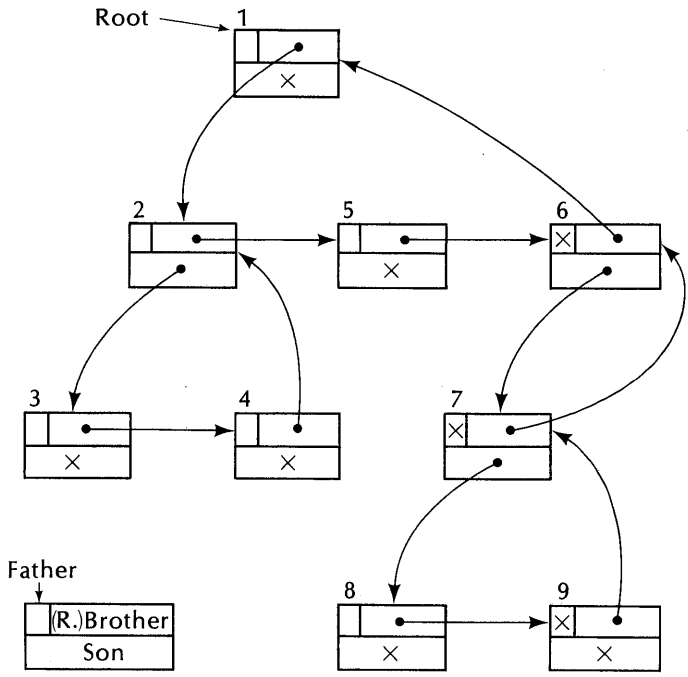

Figure 2.4. A simple pointer representation of the tree of figure 2.2.

organization correspond to a tree? What other conditions (if any) are needed?

3. Show that a tree must be acyclic, from its recursive definition.

4. Write a Pascal program that traverses a tree in preorder, given the pointer structure definition above.

Derivation Tree

A *derivation tree* displays the derivation of some sentential form in a grammar. Each node of a derivation is associated with a single terminal or nonterminal. A node associated with a terminal has no children. A node associated with a nonterminal may or may not have a set of children. Let N be a node associated with a nonterminal A, and suppose it has children. Then the children of N are associated with the symbols $x_1, x_2, \ldots, x_n$, where

$$A \rightarrow x_1\ x_2\ \ldots\ x_n$$

is a production in G.

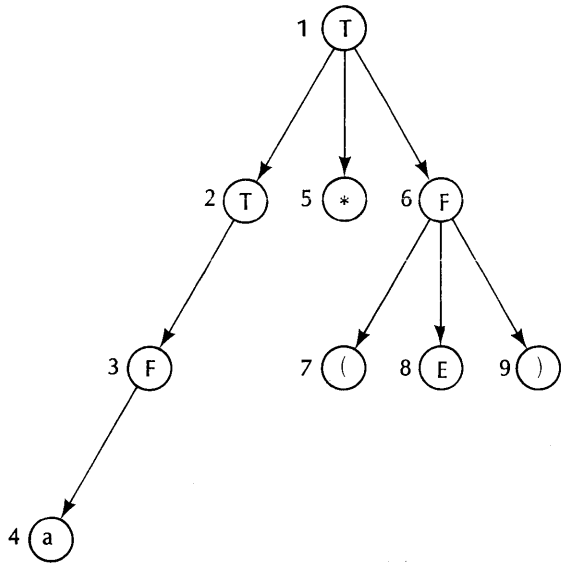

Figure 2.5. A derivation tree in grammar G_0.

Consider grammar G_0, given previously. A typical derivation tree in G_0 is shown in figure 2.5, rooted in the nonterminal element T. Its leaves, read in preorder, comprise the string

$$a*(E)$$

The string comprising the leaves of some derivation tree, in preorder, is called the *frontier* of the tree.

It is easy to show that the frontier of a derivation tree rooted in some token A is derivable from A in the tree's grammar.

We prove this by induction on the height of the tree.

Basis step. Let the height be 0. We then have $A \Rightarrow^* A$ in a derivation of zero steps, by definition of such a derivation.

Inductive step. Let the height be h, and consider some tree T of height $h+1$. Let N be the root of T. Since $h>0$, N must have at least one child. By the construction process of T, there is a production

$$A \rightarrow a_1\ a_2 \ldots a_n$$

where A is associated with N and a_i is associated with the i-th child of N. Now each of the i subtrees has a maximum height h, hence by the inductive hypothesis has a frontier f_i derivable from the token a_i. It should be clear that

the frontier of T is the left-to-right concatenation of the frontiers $f_1, f_2, \ldots, f_n$. But also

$$a_1\ a_2 \ldots a_n \Rightarrow^+ f_1\ f_2 \ldots f_n$$

Hence the frontier of T is derivable from the root token of T. QED.

The converse is also true. Given some derivation $A \Rightarrow^* w$ in a grammar G, we can always construct a derivation tree rooted in A with frontier w. The proof is left to the reader.

A picturesque way of looking at a derivation tree is to imagine that we have lots of *tree dominoes*, like the ones shown in figure 2.6. Each domino represents a production in the given grammar, and each part that carries a nonterminal is keyed so that it will only fit another domino with a matching key. The edges in each domino are made of rubber bands so that we may spread them apart as needed. We may start with any piece and build a tree downward from it. The terminal symbol parts cannot be connected to anything.

We assume that we have plenty of copies of each domino, so that we never run out of any one kind of domino.

A *complete* derivation tree for a grammar $G = (N, \Sigma, P, S)$ is such that its root node is associated with S and its frontier is a terminal string. The frontier is clearly a sentence in the language L(G). We shall normally assume that a derivation tree is complete, unless otherwise stated. A derivation tree may otherwise have a root node other than S, or its frontier may contain a nonterminal.

Exercises

1. Construct complete derivation trees for each of the following strings in $L(G_0)$:

   (a)+a
   a*(a+a)
   ((a))

2. Show that, given some derivation $A \Rightarrow^* w$ in a grammar G, there exists a derivation tree rooted in A whose frontier is w.

3. Can a derivation tree for a derivation in a context-sensitive grammar always be constructed? Why not? Give an example grammar and discuss.

4. Characterize informally the derivation tree of a right-linear grammar.

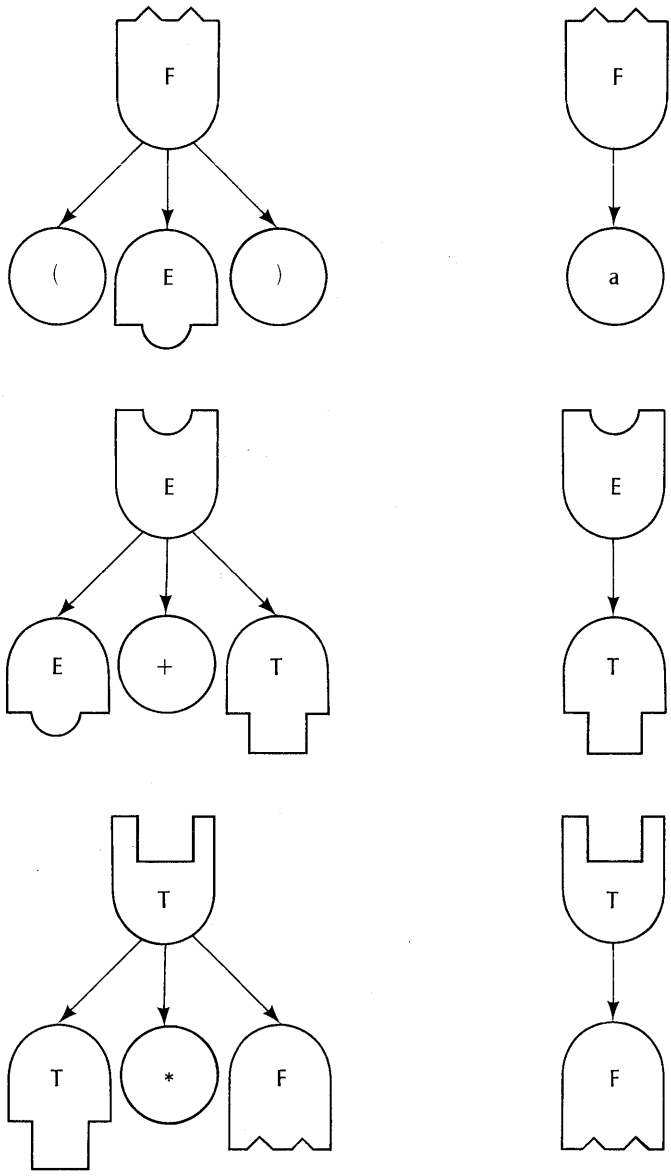

Figure 2.6. *Tree dominoes* for grammar G_0.

5. Figure 2.7 is a derivation tree for a sentence in some context-free grammar $G = \{N, \Sigma, P, S\}$ for which the productions and symbols are not known.

   (a) What is the frontier of the tree?

   (b) What symbols are necessarily in N?

   (c) What symbols are necessarily in Σ?

   (d) What productions are necessarily in P?

Syntax Trees

A *syntax* tree is a display of all of the productions in some grammar G. It contains two kinds of nodes, a *production* or P node and a *token* or T node. The root is a T node, and every path down from the root contains alternating P and T nodes.

A T node is associated with a terminal or nonterminal token A. It has children if A is nonterminal, and these are all the productions of the form $A \rightarrow w$. The children of a T node are P nodes.

A P node is associated with some production $A \rightarrow w$. Its children are the tokens in w, and these are T nodes.

Figure 2.8 shows a syntax tree for grammar G_0. T nodes are indicated by circles and P nodes by squares.

It should be clear that this defines an infinitely large tree; we can always add more nodes. However, we generally choose to consider a finite syntax tree in which each production appears exactly once. We build such a tree by starting with the root T node associated with the start symbol. We then add a production to the tree somewhere only if it does not already exist in the tree. The production consists of a P node and its T-node children. The tree's frontier then consists of T nodes.

An abbreviated represention for a syntax tree is shown in figure 2.9. Here, the P nodes have become vertical lines with horizontal branches, and the T nodes are simply the tokens of the production right part. This structure lends itself to a simple mechanical printing of a syntax tree from a set of productions.

A syntax tree may be transformed into a directed acyclic graph, called a *syntax graph* by adding directed edges as follows:

Given a nonterminal T node N with no children—it has no children because the productions normally connected to it appear elsewhere in the tree—add a directed edge from N to that T node associated with the same nonterminal symbol that does have a set of children.

Figure 2.10 shows G_0 represented as a syntax graph. We have simply added directed edges from nodes E, T, and F to their defining nodes in the tree of figure 2.9.

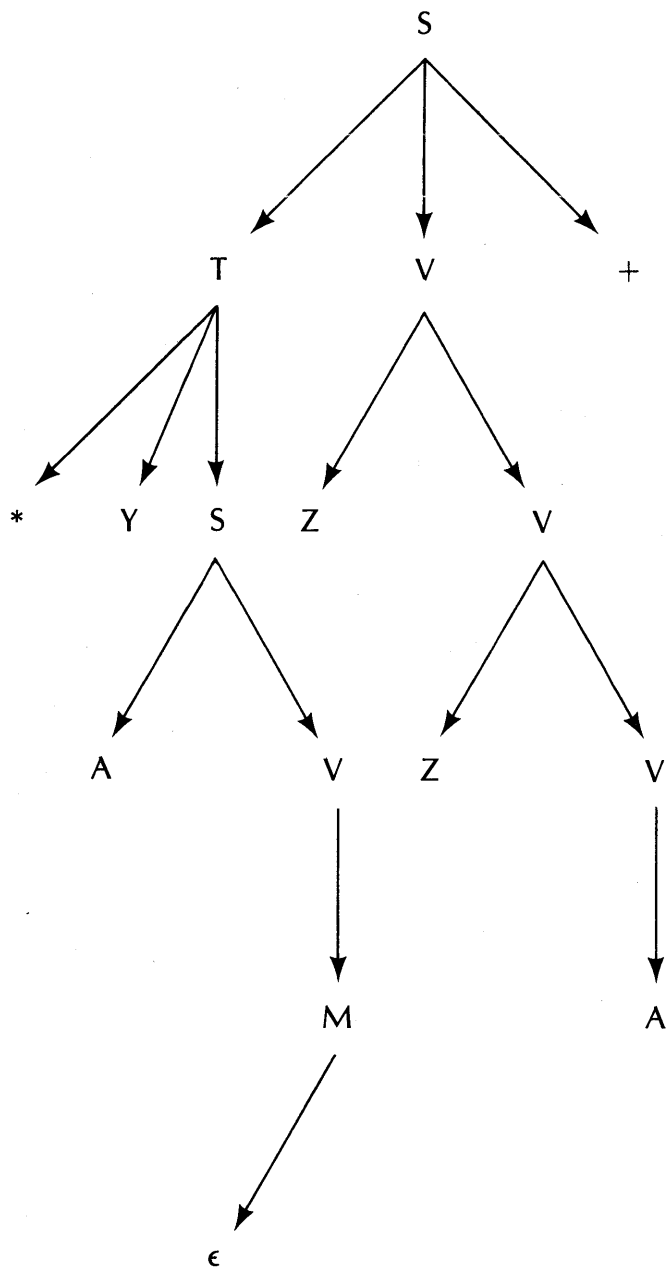

Figure 2.7. An example derivation tree.

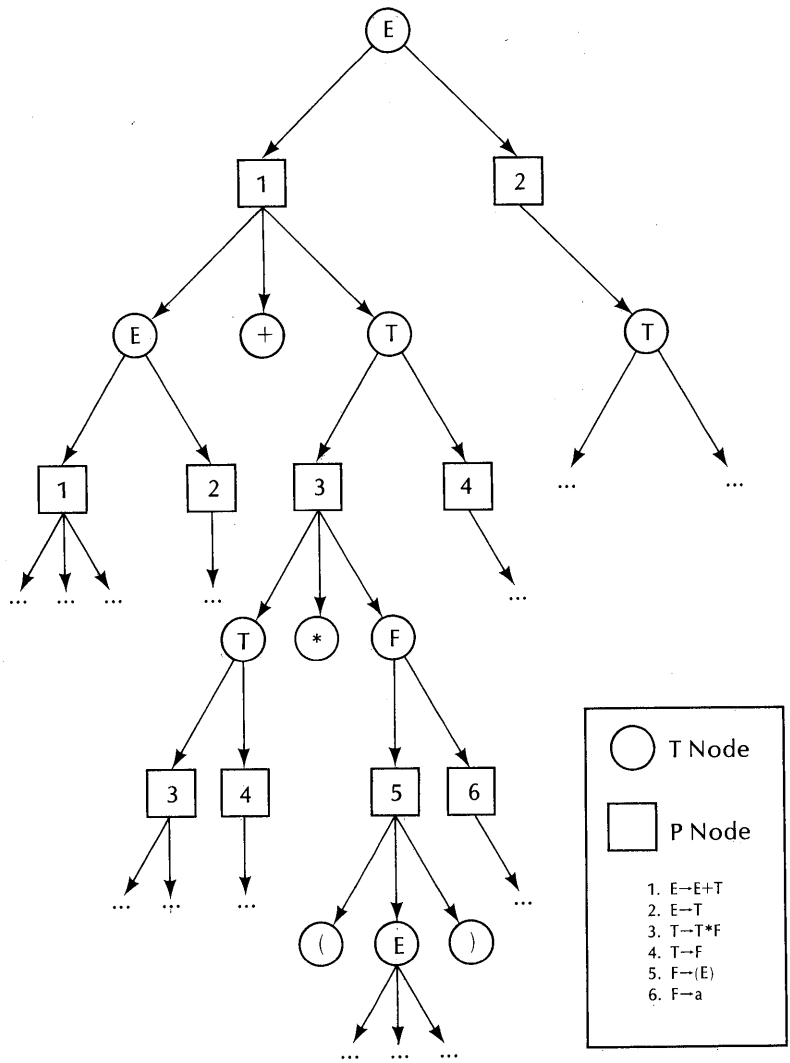

Figure 2.8. A production tree for grammar G_0.

A syntax graph is a useful and practical representation of a grammar. Cohen and Gotlieb (Cohen [1970]) show how sentences in a language may be generated or parsed by means of very simple procedures that interpret a syntax graph.

Semantic operations in a compiler must often be performed at just the right point during the sentence analysis. The operation is keyed by a particular production rule, and a syntax graph or tree enables us to easily determine the appropriate production rule for some operation.

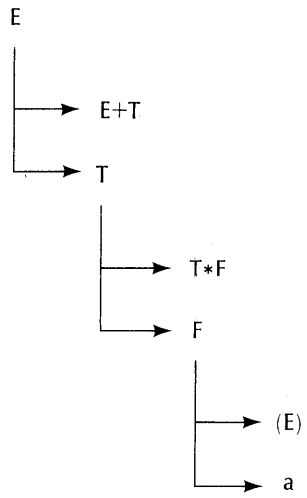

Figure 2.9. A finite production tree for grammar G_0.

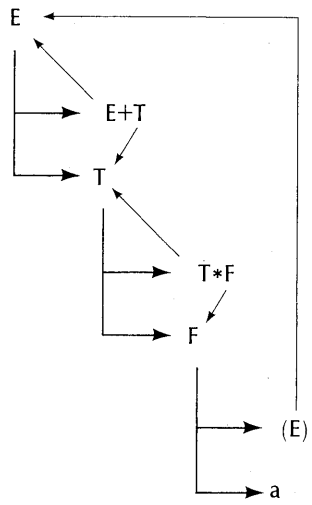

Figure 2.10. A syntax graph for grammar G_0.

Exercises

1. Consider a finite syntax tree constructed indefinitely far, but such that each T node has exactly one or zero children, and the leaves are T nodes. Show that its frontier is a sentential form.

2. Outline an algorithm that can accept a context-free grammar as input

and print its syntax tree in the form of figure 2.9. How might you deal with a finite page width and length?

3. A nonterminal X in a grammar G is said to be *useless* if and only if no terminal string can be derived from X. Develop an algorithm that identifies all useless nonterminals. (*Hint*—focus on the "useful" nonterminals; build a set of useful nonterminals; these either derive a terminal string or a string consisting of terminals and useful nonterminals in one step.) Give an algorithm for eliminating useless nonterminals from a grammar that preserves the grammar's language.

4. A nonterminal X is said to *inaccessible* if and only if no sentential form contains X. Develop an algorithm that identifies the inaccessible nonterminals and an algorithm for their elimination from the grammar.

5. Given a grammar G, possibly containing empty productions, find a transformation to an equivalent grammar G′ such that G′contains at most one empty production, $S' \rightarrow \epsilon$, where S′ is the start symbol of G′.

6. Given a grammar G, possibly containing single productions (of the form $A \rightarrow B$, where A and B are nonterminals), find a transformation to an equivalent grammar G′ such that G′ contains no single productions.

(Remark: The transformations of exercises 3 to 6 are important in reducing a grammar so that it is amenable to a precedence parsing method.)

2.2.6. Canonical Derivations

In general, a derivation step requires two kinds of choices to be made. We may have more than one nonterminal symbol in our sentential form, and for each nonterminal symbol, there usually is more than one production that may be used for the replacement.

For example, in grammar G_0, we can derive T*F as follows:

$$E \Rightarrow T \Rightarrow T*F$$

Now in the sentential form T*F, we may next replace either the T or the F. There is also more than one production with T as the left member (and with F as the left member).

The first kind of choice, that of which nonterminal to replace, has no effect on the class of sentence strings that can be derived from the start symbol. We may state this property as follows:

Given a sentential form xXyYz, where X and Y are in N, that can derive a sentence w through the derivation steps:

$$xXyYz \Rightarrow xryYz \Rightarrow^* w$$

then there exists a derivation

$$xXyYz \Rightarrow xXysz \Rightarrow^* w$$

and conversely.

Proof: In the derivation

$$xXyYz \Rightarrow xryYz \Rightarrow^* w$$

Y must be replaced somewhere. At that point we will have a sentential form

$$x'r'y'sz'$$

where $Y \Rightarrow s$, $x \Rightarrow^* x'$, $r \Rightarrow^* r'$, $y \Rightarrow^* y'$ and $z \Rightarrow^* z'$. We also know that

$$x'r'y'sz' \Rightarrow^* w$$

We may therefore reorganize the derivation as follows:

$$xXyYz \Rightarrow xXysz \Rightarrow xrysz \Rightarrow^* x'r'y'sz' \Rightarrow^* w$$

The converse is easily proven in a similar way. QED

This independence of the order of selection of nonterminals is a property of context-free grammars, but not of context-sensitive grammars. In a context-free grammar, each nonterminal can be expanded into some terminal string independently of its neighbors, and its expanded string essentially "pushes aside" its neighbors without interfering with their order in any way. Hence it doesn't matter which of several nonterminals in a sentential form are selected next for a derivation step.

We would like to have a standard derivation order, however, and each of the parsing methods to be introduced later has an inherent derivation order. Whenever we impose some ordering rule for the selection of the next nonterminal to replace in a sentential form, we have a *canonical* derivation. The two most common rules are *left-most* and *right-most*. In a left-most derivation, the left-most nonterminal in each sentential form is selected for the next replacement. In a right-most derivation, the right-most nonterminal is selected.

A top-down parse of some sentence, scanning from left-to-right through the sentence, corresponds to a left-most derivation. A bottom-up parse, scanning from left-to-right, corresponds to a right-most derivation in reverse order, i.e., the parser works from the sentence to the start symbol.

For example, in figure 2.11, we have effectively worked out the partial left-most derivation

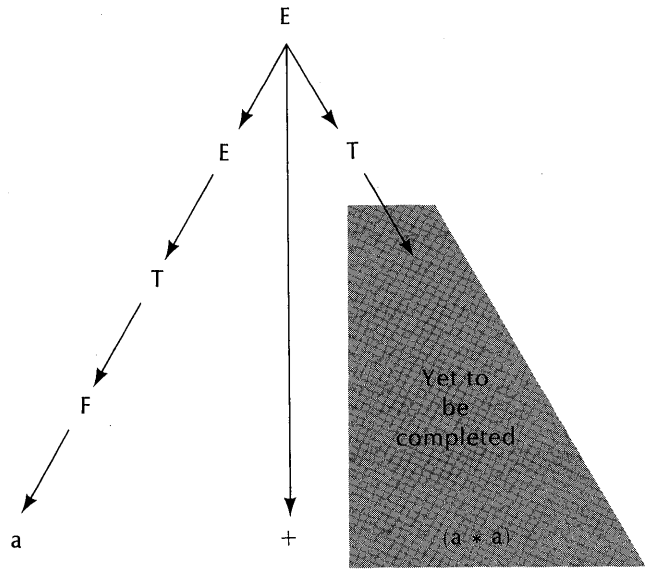

Figure 2.11. Top-down derivation tree construction.

$$E \Rightarrow E+T \Rightarrow T+T \Rightarrow F+T \Rightarrow a+T$$

The remaining parse task is clearly $T \Rightarrow^{+} (a*a)$. The next derivation step must invoke the production $T \rightarrow F$, to yield

$a+T \Rightarrow a+F$

In figure 2.12, we have effectively worked out the partial right-most derivation

$$E+(F*a) \Rightarrow E+(a*a) \Rightarrow T+(a*a) \Rightarrow F+(a*a) \Rightarrow a+(a*a)$$

A bottom-up parser has developed this in backward order, starting with $a+(a*a)$ and ending (so far) with $E+(F*a)$. The next parse step should invoke production $T \rightarrow F$ on the right-most F, so that we will have the derivation step

$$E+(T*a) \Rightarrow E+(F*a)$$

Note that this is consistent with a right-most derivation.

Exercises

1. Give right-most and left-most derivations for each of the following strings in G_0:

   a*(E) {see figure 2.5}
   (a+a)*a
   a+((a))

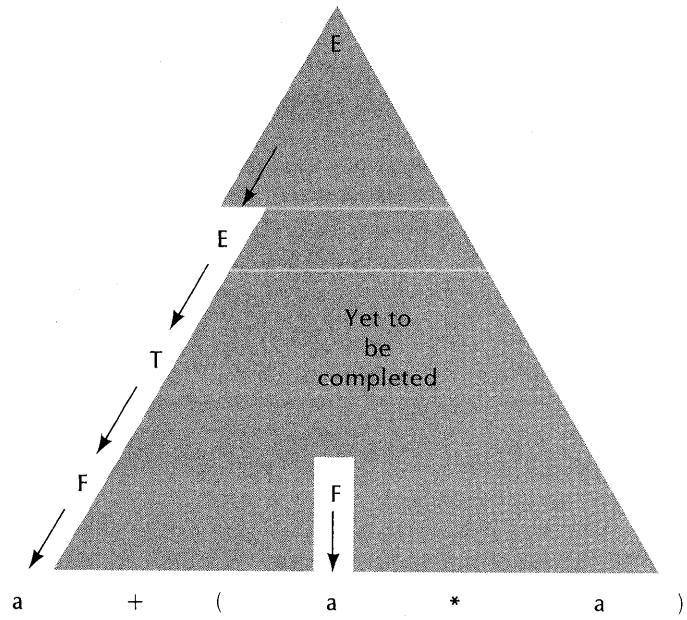

Figure 2.12. Bottom-up derivation tree construction.

2. Give yet another canonical derivation rule and illustrate a derivation in G_0 using your rule.

3. A left-most and a right-most derivation of some sentence w has the same derivation tree. Why?

2.2.7. Ambiguity

Suppose that we have a grammar and a sentence w for which two different derivation trees exist. By "different" we mean that the structure or the node labeling is different in some respect. We then say that the grammar is *ambiguous*. If no sentence has more than one derivation tree, we say that the grammar is *unambiguous*.

An ambiguous grammar is not a particularly desirable basis for a programming language. The meaning of a sentence lies mostly in its structure, as determined by the structure of its derivation tree, and not just in the set of symbols that comprise it. If there are two different derivation trees for some sentence, then it is possible that two different meanings can be attributed to the sentence.

English is full of ambiguous sentences, owing to the possibility of many words serving in different ways. For example,

"Time flies like an arrow."

can be interpreted in at least three ways: as an observation on the passage of time, as a command to compare the timing of flies with the timing of an arrow, or as a statement on the preferences of "time flies," whatever they are.

Similarly, a context free grammar may be ambiguous. For example, the grammar

$$E \rightarrow E + E$$
$$E \rightarrow E * E$$
$$E \rightarrow (E)$$
$$E \rightarrow a$$

derives exactly the same sentences as G_0, yet is an ambiguous grammar. Figure 2.13 shows two different derivation trees for the sentence

$$a+a*a$$

Now we can show that if two different derivation trees for some sentence exist, then there must also be two different canonical derivations for the sentence as well, and conversely. Thus in figure 2.13, we have the two different left-most derivations

$$E \Rightarrow E+E \Rightarrow a+E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

and $$E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow a+E*E \Rightarrow a+a*E \Rightarrow a+a*a$$

We could similarly demonstrate two different right-most derivations.

We offer a left-most derivation proof of this assertion; a right-most proof is similar.

Theorem: Two or more distinct derivation trees for some sentence w exist if and only if two or more distinct left-most derivations exist for w.

Proof—"if" part. We have two distinct left-most derivations. The two agree exactly until some derivation step, in which the left-most nonterminal is replaced by one string in one and another string in the other, e.g.,

$$S \Rightarrow^+ uXv \Rightarrow uxv \Rightarrow^* w$$

or $$S \Rightarrow^+ uXv \Rightarrow ux'v \Rightarrow^* w$$

where x and x′ are different. Now consider the two derivation trees corresponding to these derivations. They may obviously be constructed top-down by following the derivation steps. The two trees are identical until the nonterminal node X is reached; its children are the string x in one tree and x′ in the other. Yet when the construction process is complete, both trees have the frontier w. QED

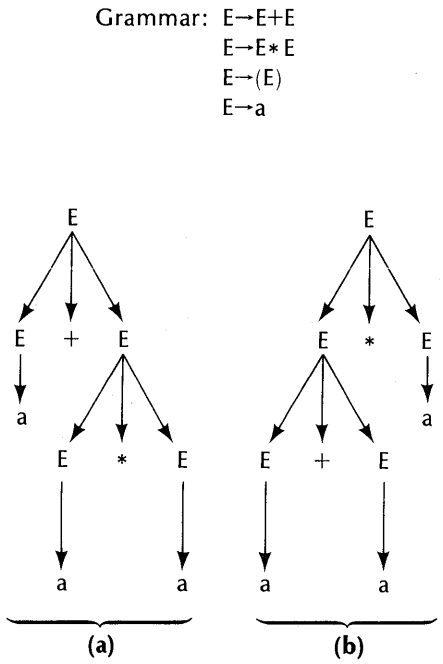

Figure 2.13. Two different derivation trees for the sentence a+a*a, through the ambiguous grammar given.

"Only if" part. Consider two different derivation trees T and T'with the same frontier w, rooted in S, and with the same grammar. We walk down through both trees (in preorder) starting at their root node, and continue as long as we find agreement, stopping on the first difference.

This tree walk is the same as the top-down construction process corresponding to a left-most derivation. If we are at some node N in T and it agrees with the corresponding node in T', then we consider the productions rooted in N and N'. If these fail to agree, we stop on node N. If they agree, then we compare each of the children, in preorder. We start the walk on the root node, and continue until the difference is found.

Now in the walk process, we have also generated two sequences of productions, corresponding to left-most derivations of the trees' frontiers. The sequences will agree until the tree difference is found, then there will be two different derivation steps, e.g.,

For tree T, we have

$$S \Rightarrow^* uXv \Rightarrow uxv \Rightarrow^* w$$

and for tree T', we have

$$S \Rightarrow^* uXv \Rightarrow ux'v \Rightarrow^* w$$

where the derivation $S \Rightarrow^* uXv$ corresponds to that portion of the tree walk just before the tree difference at node X is detected. QED

A language is said to be ambiguous if no unambiguous grammar exists for it. Note that a given language may have more than one grammar that describes it; some of these grammars may be ambiguous and others not. However, if one unambiguous grammar for a language can be found, then the language is unambiguous.

An important result in language theory states that there exists no algorithm that can accept an arbitrary context-free grammar and determine that it is either ambiguous or unambiguous. However, there exist algorithms that can return one of the results: {unambiguous, don't know}. These turn out to be parser constructor algorithms.

Exercises

These two exercises refer to the grammar $E \rightarrow E+E \mid E*E \mid a$

1. Three different derivation trees for the sentence a+a*a+a exist. Display them.

2. Suppose that ADD is emitted whenever production $E \rightarrow E+E$ is used in a left-most derivation, and MPY whenever $E \rightarrow E*E$ is used. "LOAD a" is emitted whenever $E \rightarrow a$ is used. Give the emitted code for the trees of figure 2.13, and discuss their significance.

2.3. Introduction to Parsing

A *parser* or *parsing automaton* is some system that is capable of constructing the derivation of any sentence in some language L(G) based on a grammar G. We are primarily interested only in parsers for right-linear and context-free grammars.

A parser may also be viewed as some mechanism for the construction of a derivation tree. However, we almost never actually construct a derivation tree in a practical compiler; instead the parsing algorithm makes use of a push-down stack and a finite state machine control.

Let us first look at parsing as a tree construction process.

2.3.1. Top-Down and Bottom-Up Parsing

The problem of structural analysis, or parsing, in a compiler may be seen as the problem of constructing a derivation tree, given a grammar and a sentence in the language. The sentence must form the frontier of the tree, and the tree will be rooted in the grammar's start symbol.

This is a nontrivial problem. Let us consider grammar G_0, whose productions are

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow (E)$$
$$F \rightarrow a$$

Then consider the sentence:

$$(a*(a+a))+a$$

Several different approaches may be taken. One might begin at the start symbol E and work downward toward the desired frontier. Many guesses are usually needed, and it will be found that a wrong guess somewhere in the process will usually result in an impossible situation. For example, figure 2.14 shows a partially constructed tree that appears reasonable, but in the end cannot possibly be right. Several mistakes were made in its construction. We still have to fit "a+a" into the inner parentheses, and have "+a" left over. We have found a partial tree for a sentence like (a*(a)), but it will not do for the sentence (a*(a+a))+a.

If we start at the bottom and work up, we also find ourselves making a number of guesses. It is certainly clear that every token "a" must be fitted to an F, since only one rule exists for that. It is also clear that somehow the left and right parentheses must be fitted into the production $F \rightarrow (E)$, since only that production contains parentheses. However, it is by no means clear (even with some practice) just how to fit these ideas together into a systematic plan for constructing a derivation tree.

In fact, a number of systematic derivation contruction methods have been discovered in recent years. We shall consider four of the most common and powerful of these in Chapters 4 and 5. These parsing methods fall into two broad classes—top-down and bottom-up. (A new parsing method, called *left-corner,* is an interesting blend of these two; see Rosenkrantz [1970a] and Demers [1976]).

Each of these methods reduces to the unit operation: "Determine a derivation step." Each may scan a sentence from right to left or from left to right. Now a sentence based in some grammar may be easily parsed from left to right, but with difficulty from right to left. It happens that most common programming languages are easily parsed from left to right, and furthermore, algebraic operations are usually performed in that order, by convention. We therefore confine our discussion to a left-to-right sentence scan.

Let us first consider the top-down, left-to-right parsing problem. A typical situation is shown in figure 2.11, for grammar G_0. We have already decided on the productions

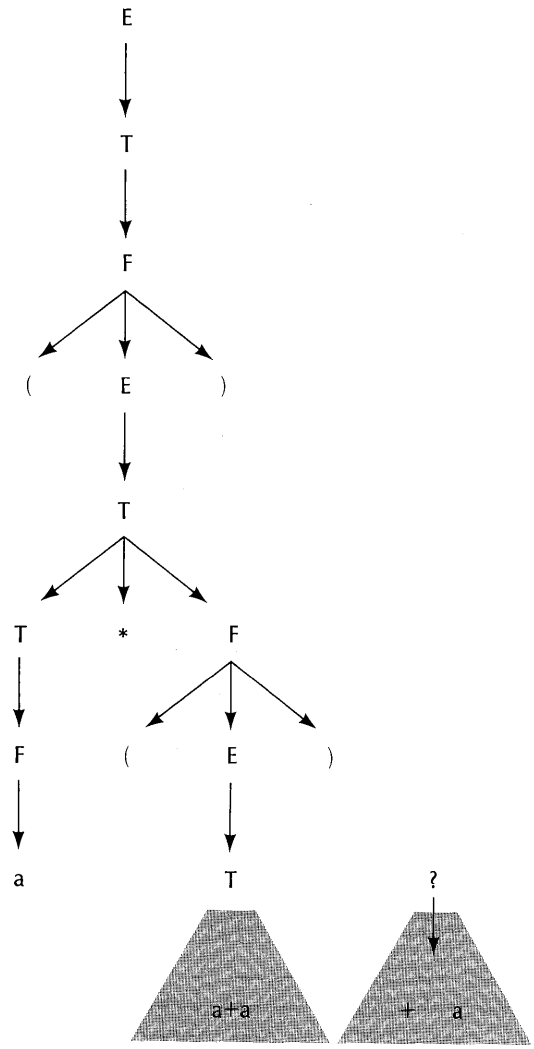

Figure 2.14. A bad guess for a derivation tree for sentence (a*(a+a))+a; top-down construction. The shaded parts cannot be incorporated in the tree.

$$E \rightarrow E+T$$
$$E \rightarrow T$$
$$T \rightarrow F$$
$$F \rightarrow a$$

and have accounted for the first two symbols a+ of the sentence a+(a*a). The left-most exposed nonterminal in the tree is T, therefore the parsing decision problem at this point may be stated:

Which of the productions $\{T \rightarrow F, T \rightarrow T*F\}$ should be connected to the exposed T node, given the partially constructed tree and the remaining input sentence (a*a)?

If we are somehow able to make the correct decision, given the information shown, each time, then we can repeat this operation again and again, until the tree is completely constructed. We would like to make each decision correctly by means of an algorithm, so that it will never be necessary to throw away our work and start over again. We also need some assurance, given any grammar, that we can find an algorithm and that it will work correctly for all the sentences in the grammar's language.

There are more considerations. What if it is possible to construct more than one derivation tree for some sentence? How can we be sure that the parsing method will reject sentences that are not in the language? These questions will be resolved when we study the top-down and bottom-up parsers in chapters 4 and 5.

Now let us consider the bottom-up parsing problem. Here we work from the given sentence upward toward the start symbol, in a left-to-right manner. We attempt to build a tree upward as far as we can before connecting productions to more sentence tokens. A partially constructed tree for grammar G_0 and the sentence "a+(a*a)" is shown in figure 2.12. We have decided that the productions

$$F \rightarrow a$$
$$T \rightarrow F$$
$$E \rightarrow T$$
$$F \rightarrow a$$

apply to the parsing process so far, and ask what the next production must be. It appears to be a more difficult decision than for a top-down parser—there are more possibilities. Should we go on to the next "a" token and apply another $F \rightarrow a$? Or should we extend the F tree through a production like $T \rightarrow F$? We can limit the choices somewhat by looking at the production right parts that can conceivably apply somewhere in the exposed tree, but in general, this still yields more choices than we can deal with.

However, we can still state the bottom-up parsing decision problem as follows:

Given a set of derivation trees (a tree may be an isolated terminal symbol or some tree rooted in a nonterminal), determine the production whose right member fits the left-most set of roots of the trees, and that "belongs" in the final derivation tree.

Neither the top-down nor the bottom-up parsing problem has a trivial or obvious solution. It is significant that these problems were not solved in a general way until about fifteen years after their statement.

2.3.2. Backtracking

The parsing problem can be seen as one of managing a sequence of choices in such a way as to find a set of choices that leads to a solution. For example, in figure 2.11, we have two choices of next production to be attached to the T node, T → T*F or T → F. Neither of these contains "(", "a" or ")". Although T → T*F contains "*", which we will need, it turns out that this choice is a poor one; the derivation tree cannot be finished if T → T*F is used at this point.

One approach to parsing is the general problem-solving method of *backtracking*. Let us first define the backtracking method in general, then show how it can be applied to top-down parsing.

Backtracking can be applied to any computation with these properties:

1. There exists a starting point and a goal.

2. The goal may be reached by starting at the starting point and following some path consisting of defined operations separated by nodes. At each node, some arbitrary choice among a finite set must be made. An operation leading from one node to another can either succeed or fail. We say the computation *blocks* if an operation fails.

3. Depending on the sequence of choices made, the computation will either reach its goal or block on some operation. If the computation blocks, we must back up one node and try another of the set of choices.

A backtracking machine will systematically explore all the choices and continue until either the goal is reached, or all the possible choices have been exhausted and lead to blocks. Unfortunately, the computation may continue forever. We need, in every application, some proof that the number of operations is bounded. We shall see that certain grammars will cause a backtracking parser to run forever on certain input strings.

There may also be more than one path to the goal. The path first found depends on the order in which the choices associated with the nodes are tried. An ambiguous grammar will yield a backtracking machine with multiple paths to the goal.

Let M be a generalized backtracking machine that contains a read/write tape used as a stack. Each cell of the tape will carry a *state* and a *choice*. Also, M manages the backtracking computation process by providing a systematic means of backing up and restarting when the process blocks.

The *process* will consist of a sequence of computations based on some algorithm, separated by choice points. At each choice point, a record is made on M's tape of the current state of the computation, and the particular choice made at that choice point. Each choice set must be finite and ordered in some

way so that it is always apparent, given a state and some choice, whether there is another untested choice.

The backtracking system then has these three moves:

1. A *forward* move from some state just recorded, using the particular choice selected by M. This will continue until: the machine blocks (step 2), it reaches its goal (halt), or it reaches another choice point (step 3).

2. A *backtrack* move, initiated by a block. Here, we consult the last-written M cell. If another choice exists in that state, we select it, record it, set the system to the state recorded in the cell, and do a forward move (step 1). If no more choices exist in that state, we remove the top cell from the tape. If the tape is empty, we halt (failure to reach goal). If the tape is not empty, we start again on step 2.

3. A *choice* move. Here we have reached a point at which some choice must be made. A new cell is added to the end of the tape containing the current system state and an *initial* choice (the first of the ordered finite set of choices available in this state). Then go to step 1.

We start the machine in step 3, by assuming that every backtracking process has an initial choice step.

In any machine application, we must show that the process will always halt in a bounded number of moves, otherwise we do not have an algorithm.

Application to Parsing a Context-Free Grammar

Let us apply our machine to the top-down parsing of a sentence in a context-free grammar. We have an input string $a_1 a_2 \ldots a_n$ of finite length. We also assume that we are building a derivation tree that will be accessible throughout the calculation. The tape will contain references to nodes in this tree.

At any point in the parse, the state of the system is the partial tree constructed so far, which incidentally includes the current position in the input string. We could conceivably just record the entire tree built so far on a tape cell, along with the particular choice of production made for the left-most exposed nonterminal node. However, such a move would be incredibly inefficient. We can accomplish the same result by storing only the two items: (1) current left-most exposed tree node, and (2) a production choice compatible with that node.

Step 1: The forward move. On a forward move, we have just chosen a production. We therefore attach it to the tree and examine the situation. Let the production be